

BRL Architecture

Changmin Park, Seongah Park, Nayoung Kim, Sungjin Park, Geonyeong Ko, Junho Lee, and Changjoo Nam

Abstract—In this paper, we present a knowledge-based POMDP framework that enables robots to selectively request human feedback under uncertainty. While modern robotic systems achieve strong autonomy, they still fail due to perception errors, execution failures, and environmental changes, often requiring continuous human supervision. We model decision making as a POMDP over a symbolic knowledge base, where uncertainty is quantified using the entropy of a belief over possible world states to determine when to query. The system also determines what to query by selecting informative queries that reduce ambiguity between competing hypotheses. We evaluate the proposed method in both a simulated waste sorting task and a real-world tomato harvesting task. Results show that our approach maintains task success rates comparable to full supervision while significantly reducing the number of queries, and user studies confirm reduced cognitive workload during interaction.

Index Terms—Robotic systems, planning under uncertainty, human-robot interaction.

I. INTRODUCTION

Recent advances in robotic systems have significantly improved autonomy in perception, manipulation, and planning across domains such as agriculture and household environments. Despite these advances, robots still encounter perception errors [1], action execution failures [2], and unexpected environmental changes [3]. As a result, many robotic systems rely on human operators to intervene if failures occur [4], [5]. However, requiring humans to continuously monitor robotic systems and determine whether robot decisions are correct is impractical in long-horizon tasks because it increases cognitive workload and fatigue [6]. Instead of relying on human operators to detect failures after they occur, robotic systems should be able to recognize uncertain situations during execution and request assistance [7], [8]. This raises two fundamental questions: (i) when should a robot request human assistance, and (ii) what information should it request?

Recent studies have explored uncertainty-aware robotic systems that selectively request human feedback during task execution [9]–[11]. These approaches demonstrate that uncertainty-aware querying can reduce unnecessary human intervention and improve decision making under uncertainty. However, uncertainty estimation, query generation, and planning are often treated as separate components. This separation prevents the robot from accounting for how the acquired feedback will actively guide downstream execution. As a result, the relationship between when to request human assistance, what information to request, and how the acquired feedback should

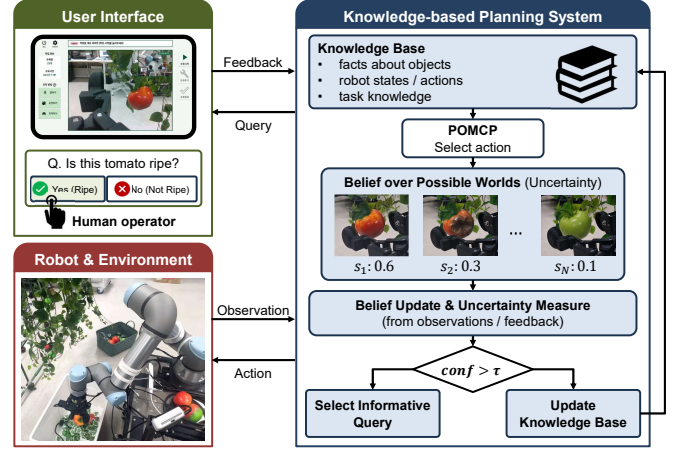


Fig. 1: Overview of the proposed knowledge-based robotic system with uncertainty-aware planning. The system maintains a belief over possible worlds and updates it based on observations. If the uncertainty exceeds a threshold, the system selectively queries the user to resolve ambiguity. Otherwise, it continues autonomous planning and execution.

influence subsequent decision making remains insufficiently explored in robotic systems operating under uncertainty.

To address this challenge, we present a knowledge-based decision-making framework that jointly incorporates uncertainty estimation, query selection, and task planning in a unified robotic architecture. We formulate decision making as a partially observable Markov decision process (POMDP) [2] and represent knowledge using a STRIPS-style symbolic model [12], [13]. The system maintains a belief over alternative world hypotheses and continuously updates this belief using observations collected during execution. The resulting belief distribution is used to determine whether additional information is required. If uncertainty exceeds a threshold, the robot requests feedback that is expected to maximally reduce ambiguity among competing hypotheses. Otherwise, the robot continues autonomous planning and execution.

To support uncertainty-aware task execution, we integrate the proposed framework into a robotic system architecture. As illustrated in Fig. 1, a centralized knowledge base connects the sensing, planning, feedback management, and user interface modules. Sensor observations and user feedback are stored in the knowledge base and shared across modules, providing a consistent representation of the environment throughout execution. Planning is performed over symbolic world states, while belief updates maintain uncertainty over alternative hypotheses. Consequently, the robot can continue autonomous execution and request human feedback only when necessary.

We validate the proposed approach in tomato harvesting and waste sorting through robot experiments and a user study. The

Changmin Park, Seongah Park, Nayoung Kim, Sungjin Park, and Geonyeong Ko are with Sogang University, Seoul, Republic of Korea.

Junho Lee and Changjoo Nam are with Institution2, Seoul, Republic of Korea.

Corresponding author: Changmin Park (email: cmp9877@gmail.com).

robot experiments compare feedback strategies by measuring task success and the number of user queries. The results show that selecting both query timing and query content reduces unnecessary queries while maintaining task success. We conduct the user study to test whether the proposed interaction design also benefits human operators during task execution. The study measures task success rate, task completion time, SAGAT accuracy, and NASA-RTLX workload. The results show that state-level queries reduce active operation time and perceived workload compared with continuous supervision or action-level queries.

The main contributions of this work are as follows:

- We propose an uncertainty-aware feedback trigger method that decides *when* the robot should ask for help.
- We introduce an information-driven query selector that decides *what* the robot should ask to reduce ambiguity among competing world hypotheses.
- We integrate uncertainty estimation, feedback requests, and symbolic planning into a single execution loop.
- We evaluate the framework in tomato harvesting and waste sorting through robot experiments and a user study.

II. RELATED WORK

In this section, we review related work along three directions. We first discuss knowledge-based robotic systems for structured environment representation. We then review planning under uncertainty, including POMDP-based and belief space methods. Finally, we examine human corrective systems that incorporate feedback for decision making.

A. Knowledge-based Robotic System

In robotics, a knowledge base refers to a structured representation of information required for decision making. These systems use logic-based representations of objects, attributes, and relations to describe the world in a way that is understandable not only by robots but also by users [64], [65]. In particular, predicate-based symbolic representations such as [13], [24] allow the system to describe both object properties and relationships among objects. In addition, graph-based models such as [66], [67] are often used to represent these relationships more clearly by organizing objects and their connections in a simple structured form.

To systematically define robot states and actions, planning languages such as PDDL [24] or FDR [25] are widely adopted. In these representations, a state is defined as a set of predicates, and each action is described by its preconditions and effects, which specify how the state changes after execution. This formulation provides a clear model of state transitions and allows robots to plan over a well-defined symbolic state space. As a result, classical planning techniques [26]–[28] can be applied to compute action sequences that achieve a given goal.

Knowledge-based robotic systems store observed facts and infer new knowledge from existing information within this symbolic representation. Logical and relational inference allows robots to identify facts that are not directly observed, combine multiple relations, and obtain plausible hidden states from contextual knowledge [14], [15]. For example, a robot

can infer the likely location of an unseen object from qualitative spatial relations, container-location relations, or place-object co-occurrence knowledge [16], [17]. Commonsense reasoning connects symbolic knowledge with perceptual evidence and supports decision making in partially observable environments [18]–[20]. Ontology structures and type hierarchies further support consistent integration of new observations and generalization across related objects, actions, and task contexts [21]–[23].

While symbolic knowledge-based systems provide structured representations and support logical inference, recent approaches have explored the use of large language models (LLMs) to further expand how knowledge is utilized in robotic planning. Early LLM-based approaches employ procedural and commonsense knowledge encoded in LLMs to propose task decompositions or action sequences. The proposed plans are then grounded with robot affordances, programmatic interfaces, or execution feedback [29]–[33]. Recent LLM-based approaches also incorporate external knowledge sources to compensate for incomplete observations. Some methods use scene graphs or open-vocabulary spatial representations to provide structured information about objects, places, and relations [34], [35]. Other methods retrieve demonstrations, procedural knowledge, or task-relevant constraints to guide long-horizon planning [36]–[40]. Related methods combine LLM-generated plans with symbolic constraints, spatial reasoning, or verification mechanisms to improve executability and consistency [41]–[45].

However, many knowledge-based robotic systems still rely on deterministic symbolic representations and assume a single consistent world state, which limits their ability to represent multiple hypotheses under uncertainty [68], [69]. As a result, such systems do not explicitly model uncertainty in action effects, sensing, or world states, and inferred facts are typically committed as definite knowledge. Recent LLM-based planning methods also suffer from grounding errors, incomplete retrieval, and hallucinated inferences, and do not maintain structured uncertainty over alternative states. While knowledge-based systems provide rich semantic structure for planning, they do not directly address uncertainty in observations, inferred facts, and knowledge validity.

To address this limitation, we maintain multiple symbolic hypotheses over possible worlds instead of committing inferred facts as deterministic knowledge. Our framework combines symbolic knowledge-based reasoning with planning under uncertainty, allowing the robot to reason over uncertain inferred states while selectively requesting human feedback if the uncertainty becomes high.

B. Planning under Uncertainty

1) *POMDP*: Unlike deterministic symbolic knowledge-based systems that assume a single world state, POMDPs explicitly reason over uncertainty through belief representations over multiple possible states. In robotic environments, such uncertainty naturally arises from sensor noise, occlusions, incomplete observations, and unexpected state changes.

TABLE I: Qualitative comparison of related works.

Related Works	Interpretable Knowledge	Symbolic Transition	Uncertainty-Aware Planning	Human Feedback	System-Initiated Query
Logic-based knowledge inference [14]–[23]	○	×	△	×	×
Classical planning [13], [24]–[28]	○	○	×	×	×
LLM-based knowledge inference [29]–[35]	○	△	×	×	×
LLM-based planning [36]–[45]	○	△	△	△	×
POMDP-based planning [46]–[54]	×	△	○	×	×
Symbolic planning under uncertainty [55]–[59]	○	○	○	×	×
Human-on-the-loop systems [60]–[63]	×	×	△	○	×
Uncertainty-aware query systems [10], [11]	△	×	○	○	○
KnowNo [9]	△	×	○	○	○
Ours	○	○	○	○	○

Sequential decision-making under these conditions has been extensively studied within the framework of POMDPs [46]–[49].

A key distinction between a POMDP and an MDP is the introduction of a belief state, which represents a probabilistic distribution over possible states when the true state cannot be directly observed. Accordingly, a wide range of solution methods have been proposed, including point-based value iteration [70], online tree search [71], and Monte Carlo tree search (MCTS) [72].

Despite these advances, effectively representing and updating the belief state remains a fundamental challenge. Since a belief is inherently a continuous distribution over the state space, exact inference is computationally difficult. Moreover, as the state space grows, the belief space expands combinatorially, making inference increasingly intractable. As a result, various approaches have been proposed to approximate the belief state.

2) *Belief Approximation*: Studies on belief approximation can be broadly categorized into particle-based and factored-based. Particle-based methods approximate the belief as a set of sampled states and have been widely adopted due to their natural compatibility with sampling-based online planning [73], [74]. For example, [72] maintains a set of particles as a history-conditioned belief and estimates action values by sampling future trajectories from a generative model without performing exact Bayesian belief updates. Similar sampling-based ideas have also been extended to large and continuous POMDPs, where DESPOT [46] evaluates a limited set of sampled scenarios and POMCPOW [75] controls search tree growth in continuous action and observation spaces.

In robotic domains, particle-based belief representations are widely used in localization, navigation, SLAM, object search, and manipulation, where particles encode hypotheses over robot, map, and object states under uncertainty. In navigation, [50] combines POMCP with model predictive

control to maximize information gain over occupancy-grid maps, while [51] uses hierarchical local online POMDP planning for exploration in large unknown environments. Similar representations have also been used for continuous control under uncertainty, including autonomous driving at occluded intersections [76] and UAV collision avoidance or target tracking under partial observability [52], [77]. In object search and manipulation, particles often represent complete scene hypotheses. For example, [53] samples visible and occluded object configurations for cluttered object search, while [54], [78] uses particle-based scene hypotheses for multi-object manipulation. Collectively, these studies show that particle beliefs provide a flexible approximation for preserving multimodal dependencies among robot, environment, and object states.

In contrast, factored belief representations model the belief by decomposing the state into a set of variables and exploiting their structure [79]–[81]. Such approaches have been widely used in structured probabilistic models, including dynamic Bayesian networks and Hidden Markov Models [82], where the state is represented in a factored form. To improve computational efficiency, various approximate inference methods have been proposed to reduce the complexity of belief updates by relaxing dependencies among variables. For example, [83] approximates belief updates by maintaining a factorized representation while partially accounting for dependencies between variables. While factored representations exploit internal structure of state space, their independence assumptions break in symbolic planning where facts are tightly linked by action constraints. Therefore, we use a particle-based approximation to preserve these dependencies.

3) *Symbolic Planning under Uncertainty*: Symbolic planning formalisms such as PDDL [24], STRIPS [13], and FDR [25] have been widely used due to their interpretability [84]. These representations are typically based on first-order logic or similar discrete formalisms [85] which describes planning problems through explicit constraints and

state transition rules. However, since traditional predicate logic only allows binary true or false values, it is difficult to directly model observation noise or state uncertainty. To address this limitation, various approaches have attempted to integrate probabilistic reasoning into logical representations. For example, [86] assign weights to logical rules to enable probabilistic inference over uncertain relationships, and [87] combine logical structure with probabilistic reasoning.

However, these approaches primarily focus on building inference models, and their high cost in model construction is unsuitable for online decision-making with complex constraints.

As a result, several studies have explored planning under uncertainty while preserving symbolic structure [55], [56]. For example, [57] encodes commonsense knowledge in ASP [85] to construct a prior over possible worlds and integrates it with probabilistic planning [88]. However, since the belief is defined over a set of possible worlds, the computational cost increases as the number of states grows. To address this, some works organize possible worlds in a structured manner. For instance, [58] construct a state space tree and a hierarchy of POMDPs based on domain knowledge, reducing complexity by leveraging abstraction. More recently, large language models have been used for belief approximation, as in [59], which builds a tree of hypotheses and uses LLMs to generate particles in open-ended settings. While these approaches enable more flexible or scalable belief representations, they either depend on predefined hierarchical structures or rely on the quality and consistency of generated hypotheses.

Prior work has explored various belief representations, and global belief approaches can be sample-intensive under strong symbolic constraints. Motivated by the broader idea of focusing computation on reachable belief regions [88], we approximate belief locally by sampling states reachable under each action. These local beliefs are organized in a tree structure, and planning is performed using the off-the-shelf planner POMCP within an interpretable symbolic space. Real-world robotic systems also benefit from information that directly disambiguates competing hypotheses, which motivates human corrective systems that selectively request feedback under high uncertainty.

C. Human Corrective System

Despite recent advances in robotic systems, robots still face operational uncertainty in dynamic and unpredictable real-world environments. Prior work has addressed this limitation by involving humans in different forms of robot autonomy [62], [63]. In particular, in human-on-the-loop (HOTL) systems, robots execute tasks autonomously, while human operators monitor the system and revise or override robot decisions if failures or errors occur [60], [61]. This supervisory structure can improve safety and robustness because human operators can compensate for errors that autonomous systems cannot resolve.

However, HOTL systems require operators to sustain attention and detect failures during robot operation. To provide effective corrective feedback, operators must first notice that

the robot is in an uncertain or erroneous state. Situation awareness research suggests that operators must then understand the current situation, project possible consequences, and determine an appropriate intervention [89]. This cognitive process can be difficult in repetitive tasks. Operators may experience vigilance decrement because continuous exposure to monotonous monitoring tasks can slow responses and reduce detection accuracy [90], [91]. Operators may also experience inattention blindness, in which visible failures are missed if attention is not directed to the relevant event [92]. As a result, continuous supervision can increase workload and fatigue [93]. Continuous supervision can also reduce the likelihood that operators detect critical robot failures [93], [94].

Prior work has shown that robots can improve task performance, team efficiency, and subjective satisfaction if they proactively provide assistance during human-robot collaboration [95]. This finding suggests that proactive robot behavior can support collaboration by reducing the need for humans to manage every decision or detect every need for intervention. In parallel, uncertainty-aware robot systems have advanced toward determining when human input is needed. For example, conformal prediction-based methods construct prediction sets with statistical guarantees and use the size of these sets to decide whether to query the user [9]–[11]. These methods show that robots can use uncertainty estimates to reduce unnecessary human intervention.

However, studies that develop and evaluate system-initiated query systems have mainly focused on technical performance, such as task performance, uncertainty calibration, and query efficiency. Human factors, such as workload and fatigue, can also affect task performance in automated systems that involve human operators [96]–[98]. Therefore, system-initiated corrective interaction should be evaluated not only in terms of technical performance but also in terms of operator burden and fatigue.

Building on this gap, we examine how system-initiated corrective interaction affects operator workload, fatigue, and task performance in automated robot systems. This motivates the user-study component of our evaluation and complements our technical contributions on belief-based feedback triggering and query selection. In our interaction structure, the robot monitors its own belief uncertainty and asks for corrective feedback when additional information is useful. Users therefore provide targeted input at system-identified moments rather than continuously monitoring for failures.

III. PRELIMINARIES

A. Symbolic State and Action Representation

The objective of planning is to find a transition sequence that transforms the initial state into a goal state. The planning problem is defined as a tuple $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, \mathcal{G} \rangle$, where $\mathcal{F}$ is a finite set of **atoms**, $\mathcal{A}$ is a set of actions, $s_0 \subseteq \mathcal{F}$ is the initial state, and $\mathcal{G} \subseteq \mathcal{F}$ is the goal condition. A state s is represented as a subset of $\mathcal{F}$. Each action $a \in \mathcal{A}$ is defined as a tuple $a = \langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$, where $\text{pre}(a), \text{add}(a), \text{del}(a) \subseteq$

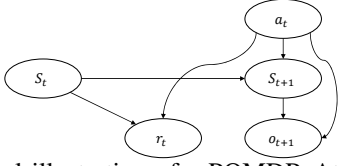


Fig. 2: Graphical illustration of a POMDP. At each time step t , the agent selects an a_t based on its current belief over the latent state s_t . The environment transitions to the next state s_{t+1} and generates a reward r_t . Since the true state is not directly observable, the agent receives an observation o_{t+1} .

$\mathcal{F}$ denote the preconditions, add lists, and delete lists of the action, respectively. An action a is said to be applicable in state s if and only if $\text{pre}(a) \subseteq s$, denoted as $a[s]$. The state transition is defined as $s \xrightarrow{a} s'$, where $s' = (s \setminus \text{del}(a)) \cup \text{add}(a)$.

A plan is a sequence of actions $\pi = (a_1, \dots, a_T)$ such that the goal condition is satisfied after sequentially applying the actions starting from the initial state, i.e.,

$$s_0 \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_T} s_T \quad \text{and} \quad \mathcal{G} \subseteq s_T.$$

B. POMDP

A POMDP is defined as a tuple $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, and Ω is the observation space. The transition model $T(s' | s, a)$ specifies the probability of reaching the next state s' after taking action a in current state s . The observation model $O(o | s', a)$ defines the probability of receiving observation o after arriving at state s' by executing action a . The reward function $R(s, a)$ gives a scalar value to each transition, and $\gamma \in (0, 1]$ is the discount factor.

Since the true state is not directly observable, the agent maintains a belief b , which is a probability distribution over $\mathcal{S}$ such that $b(s) = \Pr(s)$. After executing an action a and receiving an observation o , the belief is updated using a Bayes filter.

$$b'(s') = \eta O(o | s', a) \sum_{s \in \mathcal{S}} P(s' | s, a) b(s), \quad (1)$$

where η is a normalization constant.

A policy π maps beliefs to actions. The objective of a POMDP is to find a policy that maximizes the expected cumulative discounted reward, defined as $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t]$.

C. POMCP

Partially Observable Monte Carlo Planning (POMCP) performs Monte Carlo tree search over action-observation histories $h = \langle a_1 o_1 \dots a_t o_t \rangle$, where each node in the search tree corresponds to a history. Action selection follows the UCB rule:

$$a^* = \arg \max_a \left(V(ha) + c \sqrt{\frac{\log N(h)}{N(h)}} \right), \quad (2)$$

where $V(ha)$ is the estimated value of a after h , $N(h)$ and $N(ha)$ denote visit counts, and c is an exploration constant.

Each simulation samples the next state, observation, and reward from a generative model $(s', o, r) \sim \mathcal{G}(s, a)$. The return is computed recursively as $R = r + \gamma R'$, where R' is the return from subsequent steps. The value estimate is updated using an incremental average:

$$V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}. \quad (3)$$

The belief associated with a history h is represented implicitly as a set of particles, $B(h) = \{s | s \text{ is visited at } h\}$, which approximates the posterior over states without explicit belief updates.

IV. SYSTEM ARCHITECTURE AND METHODS

In this section, we present an overview of the system architecture and the data flows between modules. Then we give a detailed explanation of their functionalities and applications.

A. Overview

Fig. 3 illustrates the brief system architecture and the data flow between the main modules. We integrate the *Knowledge Base* (KB), the *Planning Manager* (PM), the *Sensing and Perception* module (SP), the *Feedback Manager* (FM), and the *User Interface* (UI). The KB maintains symbolic descriptions and facts used for planning, SP generates sensory information from robot sensors, PM generates and executes plans, FM coordinates corrective feedback, and UI bridges the system and the user.

The KB stores essential information required for the robot to perform tasks and explicitly represents the current state of the system. All modules share information and interact with each other through the KB. The KB is composed of three types of information: domain knowledge, static information, and local information. Domain knowledge refers to prior knowledge in a specific domain. It includes descriptions of robot skills, domain-specific rules, and general explanations of the environment. Static information is used during planning and execution but remains unchanged throughout task execution. Unlike domain knowledge, static information is encoded in a structured form and includes information such as robot parameters and local maps. In contrast, local information is used during planning and execution but cannot be known in advance. It is obtained through observations from the SP module and stored in the KB in the form of facts.

The PM is responsible for online action selection and execution under partial observability. The POMCP-based planner within the PM module selects the best action from the current belief and triggers the SP module to acquire additional observations. For instance, if the robot is positioned in front of a tomato stem, it naturally executes a `detect` action to infer the locations and ripeness states of tomatoes. To quantify uncertainty, the PM approximates the belief in the form of particle and computes the entropy over the resulting belief distribution. If such uncertainty is detected, the PM queries the FM to obtain additional information that can resolve the ambiguity of the observation. Upon receiving a response from the FM, the PM updates the belief accordingly and

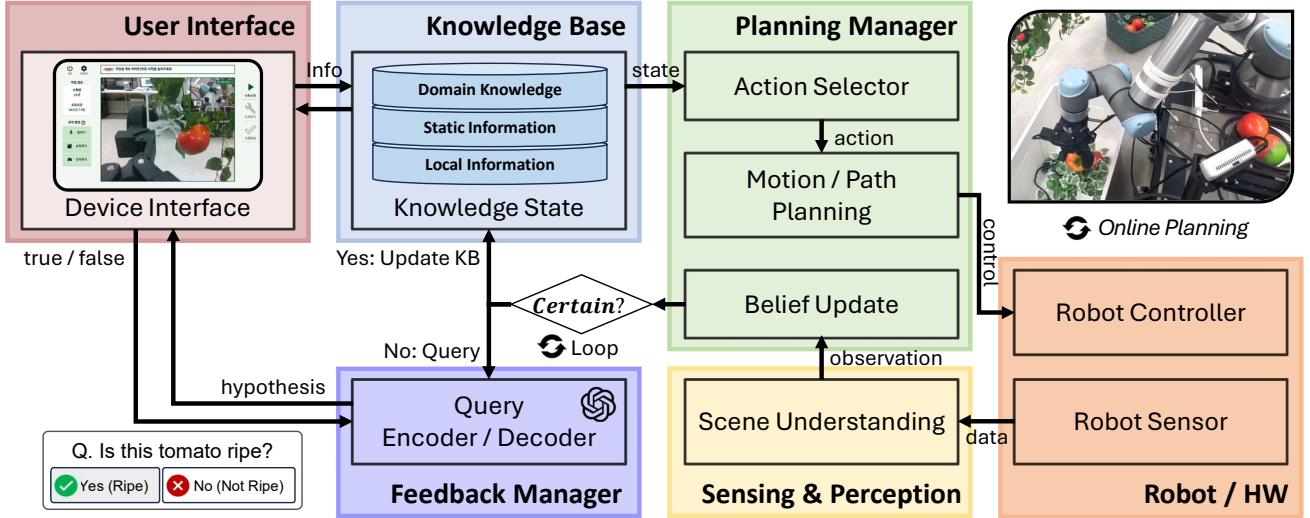


Fig. 3: System architecture and data flow of the proposed framework. The system consists of the *Knowledge Base* (KB), *Planning Manager* (PM), *Sensing and Perception* module (SP), *Feedback Manager* (FM), and *User Interface* (UI). The PM performs online planning based on the current knowledge state stored in the KB and updates the belief using observations from SP. The system evaluates the confidence of the updated belief. If the confidence exceeds a threshold, the most probable hypothesis is committed to the KB. Otherwise, the FM generates a query and requests corrective feedback from the user through the UI. The feedback is then used to refine the belief and continue planning.

promotes the most probable belief state to a certain state in the knowledge base.

The SP module provides the current visual sensing information to the UI in real time. In addition, it updates the KB with observations triggered by the PM. In this process, the SP module makes use of data from various sensors, such as cameras, LiDAR, and tactile sensors, to extract information used for task execution, including object locations, states, and properties. For example, from RGBD images, the SP module infers visual attributes such as the position, ripeness, and state of a tomato, while tactile sensors are used to perceive contact-based properties such as object stiffness or softness. The high-level perceptual information extracted by the SP module is stored in the KB in the form of symbolic facts.

The FM module converts symbolic hypotheses from the PM into user-facing questions using a predefined predicate-to-template mapping. For example, a hypothesis such as $\text{ripe}(\text{tomato}_1)$ is mapped to a question such as “Is tomato 1 ripe?”. Along with the query, the UI highlights the referenced object with a bounding box. The FM then maps the response from the user to a boolean value, which is used by the PM to filter the belief.

Finally, the UI displays the current visual sensing information and the system state, and allows the user to provide direct feedback or make selections. The user interacts with the FM module through the UI to monitor and supervise task execution.

B. Modules and Methods

1) *Knowledge Base*: The KB maintains a structured representation of the environment, including objects, properties, task-relevant relations, numeric quantities, and their associated

confidence values. Formally, we define $\mathcal{KB} = \mathcal{I}^D \sqcup \mathcal{I}^S \sqcup \mathcal{I}^L$,¹ where $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ denote the domain, static, and local information sets, respectively, as described in Sec. IV-A.

For $\mathcal{I}^S$ and $\mathcal{I}^L$, each information set contains both symbolic facts and numeric fluents. A symbolic fact is defined as $f = \text{pred}(\mathbf{x})$, where $\text{pred}(\cdot)$ denotes a predicate symbol and $\mathbf{x}$ denotes its arguments. A numeric fluent is defined as $\nu = \langle g(\mathbf{x}), v \rangle$, where $g(\mathbf{x})$ denotes a numeric function over objects and $v \in \mathbb{R}$ denotes its value. For example, symbolic facts can represent object categories and relations, such as $\text{tomato}(\text{tomato}_1)$ and $\text{loc}(\text{stem}_1)$, while numeric fluents can represent measurable quantities, such as $\text{pose_x}(\text{tomato}_1) = 0.12$ or $\text{distance}(\text{robot}, \text{stem}_1) = 2.5 \text{ (m)}$.

Specifically, the set $\mathcal{I}^S$ contains information that remains invariant throughout planning. By definition, for all $a \in \mathcal{A}$, $\text{Effect}(a) \cap \mathcal{I}^S = \emptyset$, where $\text{Effect}(a) = \text{add}(a) \cup \text{del}(a)$ denotes symbolic effects and can be extended to include numeric updates. These elements capture structural properties of the environment that are not altered by any action. For example, predicates such as $\text{tomato}(\text{tomato}_1)$ and $\text{loc}(\text{stem}_1)$, as well as fixed numeric quantities like $\text{loc_x}(\text{stem}_1) = 1.2$, remain unchanged regardless of the executed actions.

In contrast, $\mathcal{I}^L$ denotes local information obtained through the SP module and is not known a priori. This includes perceptual predicates, such as object existence or category estimates, as well as numeric fluents estimated from sensor data. For example, predicates such as $\text{ripe}(\text{tomato}_1)$ or $\text{detected}(\text{tomato}_2)$ may be inferred from observations, and numeric values such as $\text{distance}(\text{robot}, \text{stem}_1) = 2.3$ or $\text{pose_x}(\text{tomato}_1) = -0.5$ are updated as new sensor measurements become available.

¹The notation $\sqcup$ denotes a disjoint union, indicating that $\mathcal{I}^D$, $\mathcal{I}^S$, and $\mathcal{I}^L$ form a partition of $\mathcal{KB}$.

2) *Planning Manager*: The PM performs decision making based on the current belief state while continuously updating the knowledge base during execution. Uncertainty is quantified using the entropy of the belief distribution. If the uncertainty exceeds a threshold, the planner requests additional verification from the FM. Otherwise, it commits the most probable hypothesis to the knowledge base and proceeds with action execution. In the following, we describe the belief approximation, the uncertainty estimation, the POMCP based search, and the overall planning procedure.

a) *Belief representation*: In partially observable environments, the true state cannot be directly observed, and thus the PM maintains a belief, which is a probability distribution over states. The belief at time t is defined as $b_t(s) = P(s_t = s \mid h_t)$, where $h_t = \{a_1, o_1, \dots, a_t, o_t\}$ denotes the history of actions and observations. However, in symbolic domains, the state space is large and subject to structural constraints, making exact belief tracking computationally intractable. Thus, we approximate the belief using a set of particles.

Formally, at time step t , the belief is represented as a finite set of particles and their associated weights:

$$\mathcal{B}_t = \{(s_i, w_i)\}_{i=1}^N, \quad \sum_{i=1}^N w_i = 1, \quad (4)$$

where each s_i denotes a possible world composed of a set of facts. The approximated belief distribution is then

$$b_t(s) \approx \sum_{i=1}^N w_i \delta(s_i = s). \quad (5)$$

In the symbolic domain, actions exhibit partially predictable effects due to their interpretable structure. For example, `pick(tomato1)` may result in grasp failure or selecting a different tomato while leaving the robot location unchanged. Based on this observation, we define a restricted set of reachable successor states, referred to as the *frontier*. For each s and a , the frontier is defined as

$$\text{Frontier}(s, a) = \{s_1^*, s_2^*, \dots, s_j^*\}, \quad (6)$$

where each s_k^* denotes a reachable successor state.

The frontier is generated by applying the symbolic transition model of action a to the current state hypothesis. Deterministic add and delete effects are applied directly. For uncertain perceptual or execution effects, the model instantiates the mutually exclusive symbolic outcomes specified by the domain and merges duplicate successor states. Because action selection is already restricted to actions whose preconditions hold in the sampled state, each frontier contains only outcomes induced by an applicable action. Thus, the frontier size j is not a fixed hyperparameter; it is the number of feasible symbolic outcome hypotheses induced by the current action and domain model. If an action has no uncertain effect under the current state, its frontier contains a single successor.

In contrast to standard particle filtering, we assume a *knowledge* state s^K that represents the set of certain facts. Given a and s^K , we construct the belief by considering only the reachable states induced by s^K . That is, particles are

defined exclusively over the frontier $\text{Frontier}(s^K, a)$, and the belief is represented as

$$\mathcal{B}_t = \{(s_k^*, w_k)\}_{k=1}^j, \quad \sum_{k=1}^j w_k = 1, \quad (7)$$

where each s_k^* is a reachable successor state such that $s_k^* \in \text{Frontier}(s^K, a)$. After executing a and receiving an observation o , the belief over the frontier is updated by adjusting the weight of each particle according to how well it explains the observation. Specifically,

$$\forall s_k^* \in \text{Frontier}(s^K, a), \quad w'_k = w_k \cdot O(o \mid s_k^*). \quad (8)$$

Here, $O(o \mid s_k^*)$ is the action-conditioned observation likelihood defined by the SP module. It measures how likely the received observation o would be if the frontier state s_k^* were the true successor state. In the implementation, this likelihood is computed from the domain-specific observation model: for example, waste sorting uses the object-observed and category-classification probabilities for `detect` observations, while tomato harvesting uses the detection and scan observation probabilities for tomato state observations. The probabilities used in the reported experiments are summarized in Appendix B.

The complete belief-update operator used in Alg. 1 consists of frontier generation, likelihood weighting by Eq. 8, normalization of the particle weights, and removal of zero-likelihood particles. The normalized belief is then used to compute the *uncertainty* of the current state.

b) *Uncertainty of the state*: Given the updated belief over the frontier, uncertainty is quantified based on the distribution of particle weights. Intuitively, uncertainty increases when multiple candidate states have comparable likelihoods, and decreases when the belief is concentrated on a single dominant state. This uncertainty arises from *hypotheses*, which denote literals whose truth values are not yet determined.

The confidence of the current belief is defined as

$$\text{Conf}(\mathcal{B}) = 1 - \frac{\mathcal{H}(\mathcal{B})}{\log_2 j}, \quad (9)$$

where j denotes the number of particles. In the degenerate case where $j = 1$, the belief is fully concentrated on a single state and we define $\text{Conf}(\mathcal{B}) = 1$. Here, $\mathcal{H}(\mathcal{B})$ denotes the entropy of the normalized particle weights. Let $\tilde{w}'_k$ denote the normalized weights such that $\tilde{w}'_k = w'_k / \sum_{l=1}^j w'_l$. The entropy is computed as

$$\mathcal{H}(\mathcal{B}) = - \sum_{k=1}^j \tilde{w}'_k \log_2 \tilde{w}'_k. \quad (10)$$

This normalization ensures that the confidence lies in $[0, 1]$, where a uniform belief yields low confidence and a concentrated belief yields high confidence.

If the confidence exceeds a predefined threshold τ , the most probable state is selected as the updated knowledge, $s^* = \arg \max_{s_k^*} w'_k$, and the belief collapses to this committed knowledge state. The next planning step then constructs a new frontier from the committed knowledge state and the selected action. Otherwise, the system identifies uncertainty-inducing

hypotheses that differentiate candidate states.

The candidate hypotheses are constructed from literals that differ between the current knowledge state and the frontier states. Let $\mathcal{C}$ denote this set of candidate hypotheses. For each $c \in \mathcal{C}$, the belief is partitioned into two subsets depending on whether c holds:

$$\mathcal{B}_c^+ = \{(s_k^*, w_k') \mid c \in s_k^*\}, \quad \mathcal{B}_c^- = \{(s_k^*, w_k') \mid c \notin s_k^*\}.$$

The system selects the hypothesis that is expected to reduce the uncertainty the most by minimizing the expected entropy after querying:

$$c^* = \arg \min_{c \in \mathcal{C}} [P(c)\mathcal{H}(\mathcal{B}_c^+) + P(\neg c)\mathcal{H}(\mathcal{B}_c^-)], \quad (11)$$

where

$$P(c) = \sum_{s_k^*: c \in s_k^*} w_k' \quad \text{and} \quad P(\neg c) = \sum_{s_k^*: c \notin s_k^*} w_k'.$$

This one-step entropy criterion matches online execution: each query is selected at the moment the current belief is insufficiently confident, so the system asks for the hypothesis that is immediately most informative for the current frontier.

The selected hypothesis c^* is passed to the FM as a query. After receiving the answer, the belief is filtered by removing particles that contradict the feedback. If the answer confirms c^* , only particles containing c^* are retained. Otherwise, particles containing c^* are discarded:

$$\mathcal{B} \leftarrow \begin{cases} \{(s_k^*, w_k') \mid c^* \in s_k^*\}, & \text{if } ans = \text{true}, \\ \{(s_k^*, w_k') \mid c^* \notin s_k^*\}, & \text{if } ans = \text{false}. \end{cases} \quad (12)$$

The remaining weights are renormalized, and the confidence is recomputed. This process is repeated until the confidence exceeds τ or no informative hypothesis remains.

c) POMCP-based search: We employ POMCP [72] as the underlying planner for action selection. We use its sampling-based framework and focus on integrating symbolic representations with uncertainty-aware reasoning. As summarized in Alg. 2 to Alg. 4, we adapt the action set and transition sampling so that search remains consistent with the symbolic domain. The detailed pseudocode for the search, simulation, and action-selection routines is provided in Appendix A.

In Alg. 2, the search phase follows standard POMCP by repeatedly sampling a state and performing simulations. Each sampled state is passed to `SIMULATE` (line 11), and the accumulated values are used to select the final action. If the belief is empty, the search uses s^K , and at the root, the action is chosen based on value without UCB (line 13).

In the simulation phase, as described in Alg. 3, the recursive structure and return computation remain the same as in standard POMCP, but both action selection and node expansion are restricted to $\mathcal{A}(s)$ (line 10). That is, instead of considering the full action space, only actions applicable in state s are explored. If a new history h is encountered, the node is expanded only with actions in $\mathcal{A}(s)$ (line 11), and the value is initialized via a `ROLLOUT`. Here, T denotes the search tree, while $N(ha)$ and $V(ha)$ represent the visit count and value of action a , respectively. The next state, observation, and reward are then sampled from the generative

model $(s', o, r) \sim \mathcal{G}(s, a)$ (line 15). As a result, the simulation remains stochastic but is confined to reachable symbolic transitions.

In Alg. 4, the same principle is applied to both action selection and rollout. The candidate set $\mathcal{C} = \{a \mid a \llbracket s \rrbracket \in \mathcal{A}, ha \in T\}$ includes actions that are both applicable in state s and present in the tree. If `use_ucb` is true, an action is selected according to the UCB criterion defined in Eq. 2. Otherwise, the action with the highest value is chosen. During rollout, actions are sampled uniformly from all applicable actions. The generative model $\mathcal{G}(s, a)$ then produces the next state, observation, and reward, while the action space is restricted by symbolic constraints.

In symbolic domains, action preconditions and effects are explicitly defined, so exploring infeasible actions is unnecessary. Restricting the action space to $\mathcal{A}(s)$ therefore improves both search efficiency and model consistency. Overall, the structure follows POMCP while naturally incorporating the constraints imposed by the symbolic representation.

d) Action execution: Each $a \in \mathcal{A}$ selected by the POMCP is associated with an executable robot skill. The symbolic action specifies the task-level intention and parameters, while the corresponding skill defines the motion or perception routine required for execution. For example, a symbolic action such as `navigate(robot, l1, l2)` is mapped to a navigation routine, and `pick(robot, tomato1)` is mapped to a predefined manipulation trajectory.

In this work, $\forall a \in \mathcal{A}$ follows an encoded execution policy that maps a symbolic operator to a robot skill through an off-the-shelf motion planner, such as MoveIt [99]. The planner takes symbolic facts and numeric fluents stored in the KB as inputs. Numeric fluents, such as object poses, distances, and grasping parameters, define the parameters of motion generation and maintain consistency between high-level planning and low-level execution.

Specifically, after each action execution, the system receives an observation o from the environment and uses it for belief update. The system uses these observations to update both symbolic predicates and numeric fluents in the KB. This design separates task-level decision making from low-level motion planning. The KB provides a shared representation across these levels. As a result, the PM does not depend on a specific motion planner implementation.

e) Overall planning pipeline: We adopt an online planning framework as described in Algorithm 1. The planning process starts from the initial knowledge state s_0^K , which is used to initialize the belief (line 4). At each iteration, the planner first selects a_t by performing Monte Carlo tree search over the current belief $\mathcal{B}$ (line 7). If no applicable action exists, the process terminates. Otherwise, the selected action is executed and an observation o_{t+1} is received from the environment.

The belief is then updated using the observation model (line 11), and the confidence of the updated belief is computed (line 12). If the confidence is below the threshold τ , the planner queries the FM to reduce uncertainty. Specifically, it selects the most informative hypothesis, sends it to the FM, and filters out particles inconsistent with the answer (lines 14–16).

Algorithm 1: Planning Pipeline

```

1 Input: initial knowledge  $s_0^K$ , confidence threshold  $\tau$ 
2 Output: plan  $\pi$ 
3 procedure  $\text{PLANNING}(s_0^K)$ 
4    $\text{knowledge}(\mathcal{B}) \leftarrow s_0^K$ 
5    $t \leftarrow 0$ 
6   repeat
7      $a_t \leftarrow \text{SEARCH}(\mathcal{B})$ 
8     if  $a_t = \emptyset$  then
9       // There is no  $a[s]$ . (Dead-end)
10      break
11     execute  $a_t$  and receive observation  $o_{t+1}$ 
12     // UPDATE performs frontier
13     // weighting and normalization
14      $\mathcal{B} \leftarrow \text{UPDATE}(\mathcal{B}, a_t, o_{t+1})$ 
15      $c \leftarrow \text{CONF}(\mathcal{B})$  (Eq. 9)
16     while  $c \leq \tau$  do
17        $\text{hypo}^* \leftarrow \text{SELECT-HYPO}(\mathcal{B})$  (Eq. 11)
18        $\text{ans} \leftarrow \text{QUERY-FM}(\text{hypo}^*)$ 
19        $\mathcal{B} \leftarrow \text{FILTER}(\mathcal{B}, \text{hypo}^*, \text{ans})$  (Eq. 12)
20        $c \leftarrow \text{CONF}(\mathcal{B})$ 
21        $s^K \leftarrow \arg \max_{s_k \in \mathcal{B}} w'_k$ 
22        $\text{knowledge}(\mathcal{B}) \leftarrow s^K$ 
23        $t \leftarrow t + 1$ 
24   until  $t = \text{MAX-STEP}$ ;
25   return  $\pi = (a_0, \dots)$ 

```

This process is repeated until the confidence exceeds τ . Once the belief becomes sufficiently confident, the most probable state is committed as the updated knowledge state, and the planner proceeds to the next step.

3) *Sensing and Perception:* The SP module acquires raw sensory data, generates local information in $\mathcal{I}^L$, and updates the numeric fluents in the KB. It also provides observations required for belief updates during action execution. Let $\mathcal{M}$ denote the set of perception models. For each predicate pred associated with local information in $\mathcal{I}^L$, there exists a corresponding perception model $m_{\text{pred}} \in \mathcal{M}$ defined as

$$m_{\text{pred}} : \mathcal{R} \rightarrow \mathcal{I}^L,$$

where $\mathcal{R}$ denotes the space of raw sensory data. Given a sensory input $r \in \mathcal{R}$, the model $m_{\text{pred}}(r)$ produces a symbolic fact $f = \text{pred}(\mathbf{x})$ or a numeric fluent $\nu = \langle g(\mathbf{x}), v \rangle$ in $\mathcal{I}^L$.

Through this process, perceptual predicates such as $\text{ripe}(\text{tomato}_1)$ or $\text{detected}(\text{tomato}_2)$ are instantiated, and numeric fluents such as $\nu = \langle \text{distance}(\text{robot}, \text{stem}_1), 2.3 \rangle$ or $\nu = \langle \text{pose_x}(\text{tomato}_1), -0.5 \rangle$ are continuously updated based on incoming observations. Since $\mathcal{I}^L$ represents information that is not known a priori, all elements in this set are generated and refined exclusively through the SP module.

In addition, the SP module defines the observation interface of the system. After executing an action $a \in \mathcal{A}$, the system receives an observation $o \in \mathcal{O}$ derived from the current sensory input, which is used to update the belief over possible



Fig. 4: User interface for corrective feedback. The tablet interface presents the queried object and response options used to update the belief.

states. Furthermore, if triggered by the PM, the SP module acquires new sensory input and evaluates the relevant perception models to produce observations and update the associated elements in $\mathcal{I}^L$.

4) *Feedback Manager:* As illustrated in Fig. 4, the FM presents hypotheses selected by the PM and converts user responses into a form that can be processed by the system. Each queried hypothesis is represented internally as a predicate, such as $\text{ripe}(\text{tomato}_1)$. The FM maps this predicate to a predefined question template, such as “Is tomato 1 ripe?”. To support user understanding, the queried object is visualized with a bounding box indicating its location.

Users can respond via touch or speech, and the FM converts these responses into a boolean form (true or false) that can be used for belief updates. This design keeps the feedback interface simple: the planner selects what predicate to verify, while the FM only formats the selected predicate and records the user’s answer.

5) *User Interface:* As shown in Fig. 4, the UI enables interaction between the user and the system and is implemented on a tablet. It presents queries generated by the FM and overlays the queried object on the camera image using a bounding box to support intuitive understanding. Users provide responses through simple touch input, with optional speech input. The interface hides internal representations and is designed to elicit accurate feedback with minimal workload.

V. EXPERIMENTS

Our experiments examine the proposed system through two complementary layers. In the system evaluation, an oracle user always provides correct answers to robot queries, and we measure task success, execution length, and query frequency as properties of the planning framework. We first analyze how the confidence threshold τ affects these measures. We then compare four feedback strategies: **All**, **No**, **Random**, and **Ours**. The user study examines how the resulting query styles shape user performance, workload, and situation awareness with actual participants.

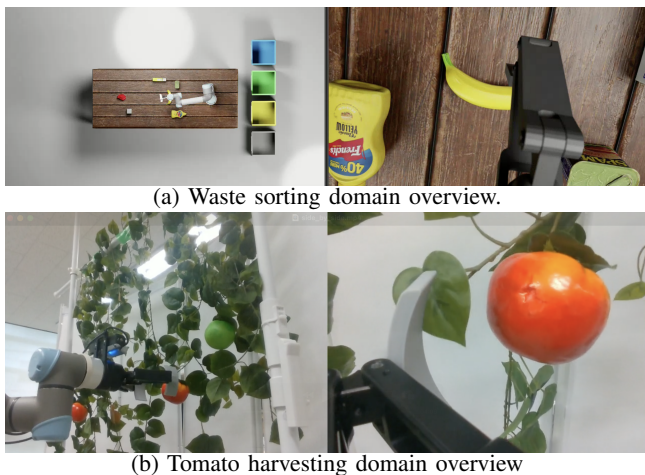


Fig. 5: The illustration of the environments. (a) A UR5 robot sorts four types of waste into designated bins in Isaac Sim. (b) A mobile manipulator harvests ripe tomatoes and discards rotten ones in a real-world setting.

A. Experimental Setup

The experimental setup includes a simulated waste sorting domain and a real-world tomato harvesting domain. For waste sorting, NVIDIA Isaac Sim provides a tabletop environment with a fixed UR5 robotic arm for pick-and-place tasks. For tomato harvesting, a Summit mobile platform carries a UR5e robotic arm and operates in a lab environment. The mobile platform uses an RGB-D camera for visual perception and a LiDAR sensor for localization. Both domains use YOLOv12 for object detection [100].

1) *Waste sorting domain:* As shown in Fig. 5a, a fixed UR5 robot is positioned between the objects and the recycling bins and performs three actions: `detect`, `pick`, and `place`. We assume that all objects are reachable by the manipulator.

Each object belongs to one of four categories: `general`, `paper`, `can`, and `plastic`. The object categories are not known a priori and must be inferred through perception. Since objects may be partially occluded by others, a single observation may not reveal all categories, requiring repeated detection during execution. The `detect` action can observe multiple objects simultaneously, after which the robot places each object into the corresponding recycling bin.

The initial arrangement of objects is randomized in every episode. A task is considered successful when all objects are correctly sorted within the maximum step limit. A reward of +5 is assigned when a waste object is placed into the recycling bin corresponding to its category, and an additional reward of +10 is given when all objects are correctly sorted. All other actions receive zero reward.

2) *Tomato harvesting domain:* We consider a tomato harvesting scenario in which four tomatoes are attached to two stems, denoted as `stem_1` and `stem_2`, as shown in Fig. 5b. The robot navigates between three locations: `dock_station`, `stem_1`, and `stem_2`, and performs `detect`, `pick`, `scan`, `place`, and `discard` actions.

Each tomato can be in one of three states: `ripe`, `unripe`, or `rotten`. The tomato states are initially unknown and must be estimated through perception. During detection, ripe and rotten tomatoes are visually ambiguous and cannot always be reliably distinguished. To resolve this ambiguity, the robot performs a `scan` action after picking a tomato to verify its condition before harvesting or discarding it. Unripe tomatoes are left untouched.

The tomato locations are randomized across stems in every episode. A task is considered successful when all ripe tomatoes are harvested and all rotten tomatoes are discarded within the maximum step limit. A reward of +5 is assigned when a ripe tomato is correctly harvested or a rotten tomato is correctly discarded after scanning. An additional reward of +10 is given when all task-relevant tomatoes are processed correctly. All other actions receive zero reward.

3) *Implementation Details:* In all experiments, the planner used a discount factor $\gamma = 0.95$, UCB exploration constant $c = 1.0$, maximum simulation depth $depth_{\max} = 20$, and $TIMEOUT = 100$ Monte Carlo simulations per planning step. Each episode was limited to 50 steps. Each reported domain-threshold or domain-strategy result aggregates five scenario configurations with 200 randomized episodes per scenario, yielding 1000 episodes per domain-condition cell. Each episode was initialized with a pseudorandom seed that controls both Python `random` and NumPy sampling and is stored with the raw execution log.

The threshold-sweep experiment treats τ as the independent variable and is reported first in Sec. V-B1. After observing that performance saturates for $\tau \geq 0.7$ while query frequency continues to increase, we use $\tau = 0.8$ for the subsequent strategy-comparison experiment.

B. System Evaluation

We evaluate the proposed framework from three perspectives. We first examine how the confidence threshold τ influences the trade-off between task performance and user feedback. We then compare the proposed query strategy with representative baselines under identical task settings. Finally, we analyze the computational scalability of the method as the symbolic state space increases.

1) *Threshold Ablation:* We first evaluate the effect of the confidence threshold τ on system performance. Fig. 6 and Table II report the resulting success rate, execution steps, and number of user queries across different threshold values.

The results show that the two domains respond differently to the confidence threshold. In waste sorting, performance improves rapidly once a small amount of uncertainty is resolved, increasing from 11.5% at $\tau = 0.1$ to 82.0% at $\tau = 0.2$. Because task success primarily depends on identifying the correct waste category, additional feedback provides limited benefit once the category is determined. In contrast, tomato harvesting involves a longer sequence of dependent decisions, including navigation, detection, picking, scanning, and execution. Unresolved uncertainty can therefore propagate through multiple stages of the task, causing performance to remain low at small threshold values. Success increases substantially only at higher thresholds and reaches 82.5% at $\tau = 0.8$.

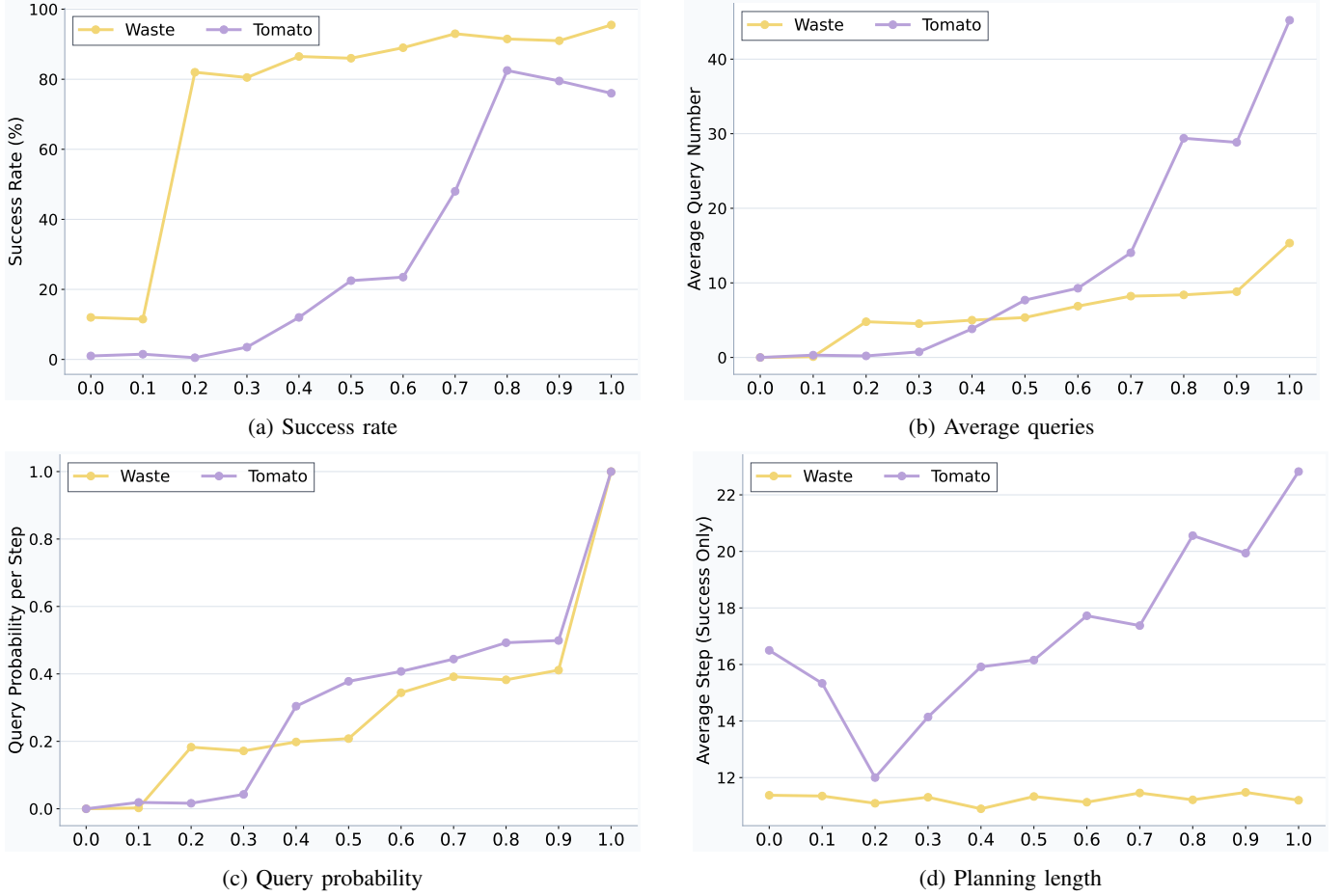


Fig. 6: Effect of the confidence threshold τ on system behavior. (a) Task success rate. (b) Average number of questions per episode. (c) Query probability per execution step. (d) Planning length for successful runs. Success rate increases as the system asks more questions, but the benefit saturates after the high-confidence regime is reached.

TABLE II: Threshold sensitivity for representative values of τ . Success rates are percentages. Query counts are average questions per episode, query rate is the fraction of execution steps with at least one question, and average steps are computed over successful runs.

τ	Waste Sorting				Tomato Harvesting			
	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps
0.0	12.0	0.0	0.00	11.4	1.0	0.0	0.00	16.5
0.2	82.0	4.8	0.18	11.1	0.5	0.2	0.02	12.0
0.6	89.0	6.9	0.34	11.1	23.5	9.3	0.41	17.7
0.8	91.5	8.4	0.38	11.2	82.5	29.4	0.49	20.6
1.0	95.5	15.3	1.00	11.2	76.0	45.2	1.00	22.8

Increasing τ consistently increases interaction because the planner requests feedback for a larger fraction of uncertain beliefs. This trend is reflected in both the average number of queries and the query rate, which increase monotonically with the threshold. However, the corresponding gains in task success eventually saturate. For example, in tomato harvesting, increasing the threshold from $\tau = 0.8$ to $\tau = 1.0$ increases the average number of queries from 29.4 to 45.2 and the query rate from 0.49 to 1.00, while producing a comparable level of task performance. The average planning length also remains relatively stable across threshold values, suggesting that additional feedback primarily improves belief quality rather

than reducing the number of actions required to complete the task. Together, these results indicate that selectively resolving task-relevant uncertainty can maintain task performance while reducing interaction cost. Based on this trade-off, we use $\tau = 0.8$ in the remaining experiments.

2) *Baseline Comparison:* We next compare the proposed query strategy with representative alternatives. **All** requests query feedback whenever multiple frontier hypotheses remain, regardless of the current confidence level. **No** performs autonomous execution without user feedback. **Random** selects queries uniformly at random. **KnowNo** [9] is a GPT-4o-based action-query baseline [101]. In contrast, **Ours** selectively

requests state-level feedback based on belief confidence and expected information gain. All methods are evaluated under identical task settings using $\tau = 0.8$.

We note that **All** is not equivalent to setting $\tau = 1.0$. The **All** policy requests feedback whenever multiple frontier hypotheses remain, whereas $\tau = 1.0$ continues querying until complete confidence is achieved. Consequently, **All** may terminate querying even when uncertainty remains, while $\tau = 1.0$ attempts to eliminate all uncertainty before acting.

All achieves high success rates in both domains. In waste sorting, it obtains the highest success rate of 97.5%. In tomato harvesting, it achieves a success rate of 81.5%. These gains come at the cost of frequent interaction, requiring an average of 16.1 and 48.6 queries in the two domains, respectively. The query rate reaches 1.00 in both domains because feedback is requested whenever a query is available. Despite this intensive supervision, the average number of execution steps remains comparable to the other methods, suggesting that additional feedback mainly improves decision quality rather than shortening the task itself.

As expected, **No** performs poorly in both domains, achieving only 13.5% success in waste sorting and 1.5% in tomato harvesting. Without user feedback, incorrect observations remain uncorrected and propagate through subsequent symbolic decisions. **Random** improves over **No** in waste sorting, increasing the success rate from 13.5% to 84.0%. However, its performance drops substantially in tomato harvesting, where it achieves only 11.0% success despite requiring 18.9 queries on average and maintaining a query rate of 0.58. This result suggests that query frequency alone is insufficient. Randomly selected questions often fail to resolve the uncertainty that directly influences future decisions.

In contrast, **Ours** provides the best balance between task success and interaction cost. In waste sorting, it achieves a success rate of 91.5% while requiring nearly half as many queries as **All**. In tomato harvesting, it slightly outperforms **All**, achieving 84.5% success while reducing the average number of queries from 48.6 to 28.8. The query rates of 0.38 and 0.48 further indicate that only a subset of uncertain decision points require human feedback. The average number of execution steps remains comparable to **All**, demonstrating that the reduction in interaction does not sacrifice task efficiency. These results indicate that feedback is most effective when it is requested only for low-confidence beliefs and focused on uncertainty that directly affects planning.

KnowNo exhibits a different trade-off. It achieves the lowest query rates among all feedback-based methods, requiring only 2.6 and 2.9 queries on average in the two domains. It also records the shortest successful execution sequences, with average planning lengths of 10.0 and 14.0 steps in waste sorting and tomato harvesting, respectively. Consequently, the overall amount of user interaction remains substantially lower than both **All** and **Ours**. Unlike state-level feedback, action-level feedback directly influences the planning process. By selecting the next action, the user provides information while simultaneously guiding future execution. This guidance reduces both planning length and query frequency, leading to fewer subsequent queries and shorter execution trajectories.

However, this efficiency does not translate into reliable task completion. The success rates remain limited to 38.5% and 42.0%. Although action-level guidance can efficiently steer execution, it does not consistently resolve the underlying state uncertainty. Users must choose among candidate actions without direct access to the belief state of planner, whereas state-level questions focus on observable facts that can be verified directly. As a result, **Ours** requires more interaction than **KnowNo** but achieves substantially higher task success through more reliable belief updates.

3) *Scalability Test*: We next examine whether the proposed framework remains effective as task complexity increases. The original setting contains four objects per domain. The scaled setting contains six objects in both domains, increasing the symbolic belief space, the number of possible object assignments, and the planning complexity. We keep the policy fixed to **Ours** with $\tau = 0.8$ and increase the maximum planning horizon to accommodate the longer task sequences.

The scaled waste sorting results show that the proposed framework remains effective as the number of objects increases from four to six. The success rate remains nearly unchanged at approximately 92.0%, while the average plan length increases from 11.2 to 15.3 steps. The average number of queries also increases from 8.3 to 11.3. However, the query rate decreases slightly from 0.38 to 0.33, indicating that the additional objects primarily increase task length rather than the fraction of decision points that require feedback.

The tomato harvesting domain presents a more challenging scalability test because uncertainty is distributed across multiple state variables, including object identity, stem location, ripeness, scan outcomes, and grasped objects. As the number of tomatoes increases, the success rate decreases from 84.5% to 74.0%, while the average plan length increases from 19.7 to 33.1 steps. The average number of queries also rises from 27.3 to 80.6. Despite this increase, the query rate changes only moderately from 0.48 to 0.54. This result suggests that the policy continues to request feedback selectively rather than querying at every decision step. The primary scalability cost appears in planning and belief maintenance, as reflected by the increase in episode execution time from 56.0 to 159.9 seconds.

Overall, the scalability results suggest that the proposed policy generalizes to larger symbolic scenes while preserving its selective feedback mechanism. As task complexity increases, the method requires more interaction in the harder tomato domain, but the query rate increases only moderately and does not collapse into dense supervision.

4) *System Discussion*: The system evaluation highlights two key questions in human-robot interaction. The first is *when* the robot should ask for feedback. The second is *what* information it should request. The results suggest that both decisions are important for maintaining task performance while reducing user interaction.

The threshold experiment shows that more feedback is not always better. Low thresholds leave important uncertainty unresolved and lead to poor task performance. High thresholds increase the number of questions but do not provide proportional gains in success rate. The best performance is obtained

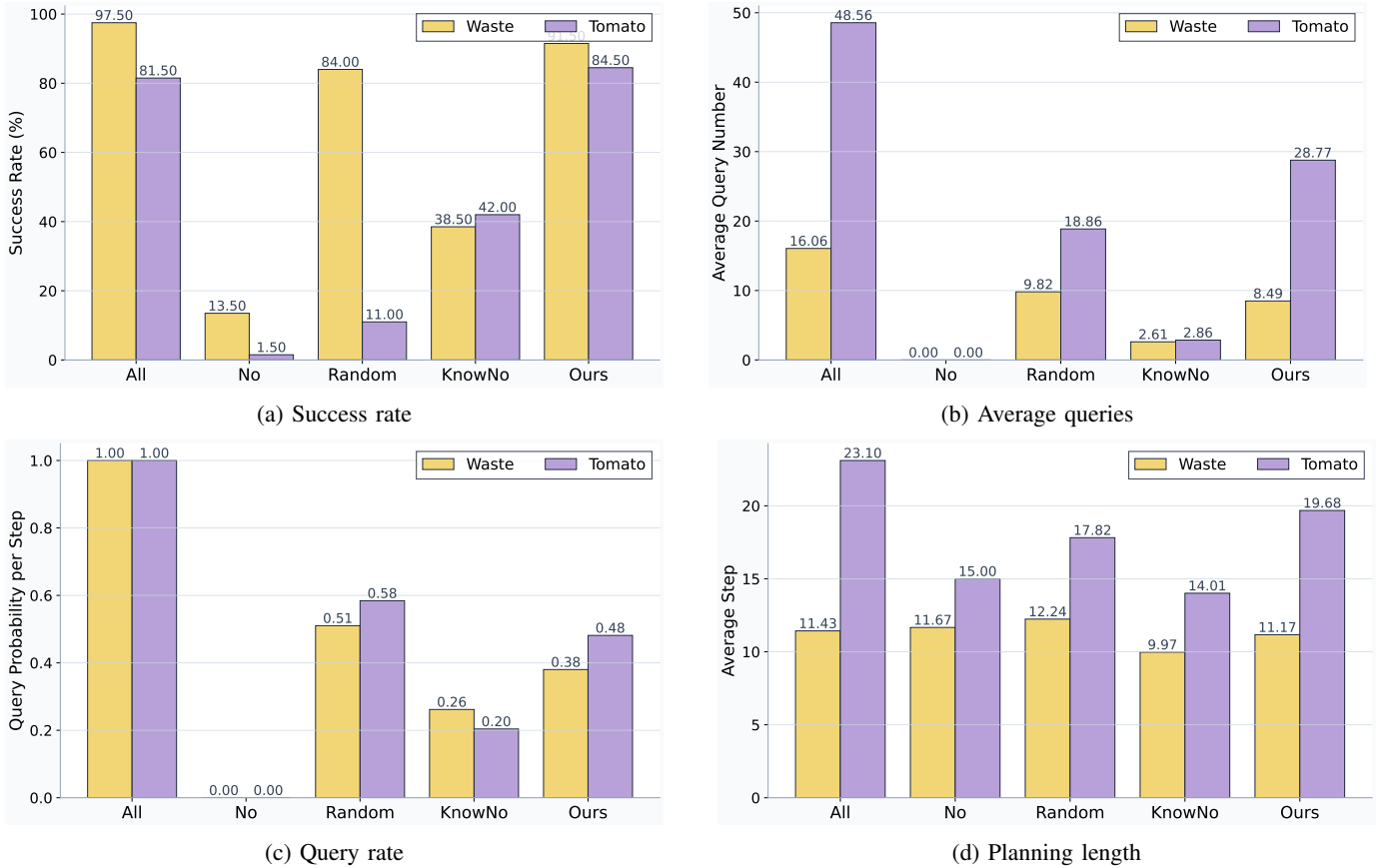


Fig. 7: Baseline comparison at $\tau = 0.8$. (a) Task success rate. (b) Average number of questions in successful runs. (c) Query rate, computed as the fraction of successful-run execution steps in which at least one question is asked. (d) Planning length for successful runs. The proposed policy preserves high task success while asking substantially fewer questions than dense supervision.

TABLE III: Baseline comparison at $\tau = 0.8$. Average queries, query rate, and average steps are computed over successful runs to compare the interaction cost of completed tasks.

Condition	Waste Sorting				Tomato Harvesting			
	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps	Success Rate (%)	Avg. Queries	Query Rate	Avg. Steps
All	97.5	16.1	1.00	11.4	81.5	48.6	1.00	23.1
No	13.5	0.0	0.00	11.7	1.5	0.0	0.00	15.0
Random	84.0	9.8	0.51	12.2	11.0	18.9	0.58	17.8
KnowNo	38.5	2.6	0.26	10.0	42.0	2.9	0.20	14.0
Ours	91.5	8.5	0.38	11.2	84.5	28.8	0.48	19.7

TABLE IV: Scalability results for **Ours** on six-object scenes.

Metric	Waste		Tomato	
	Orig.	Scale	Orig.	Scale
Success (%)	91.5	92.0	84.5	74.0
Avg. steps	11.2	15.3	19.7	33.1
Avg. queries	8.3	11.3	27.3	80.6
Query rate	0.38	0.33	0.48	0.54
Episode time (s)	5.3	22.9	56.0	159.9

when feedback is requested only for uncertainty that is likely to affect future planning decisions.

The baseline comparison shows that asking the right question is as important as deciding when to ask. **All** achieves

high success rates but requires frequent interaction. **Random** also asks many questions, yet often fails because the selected questions are unrelated to the uncertainty that drives the task. **KnowNo** reduces both planning length and interaction by allowing users to directly guide action selection. However, action-level guidance does not reliably resolve the underlying state uncertainty, resulting in substantially lower task success. In contrast, **Ours** focuses on state uncertainty that directly affects planning and requests feedback only when that uncertainty becomes important. This allows the system to maintain high success rates while avoiding unnecessary questions.

The scalability experiment shows that larger problems primarily increase planning and belief maintenance costs. In the waste sorting domain, performance remains stable as the

number of objects increases. In the tomato harvesting domain, the larger belief space leads to longer planning horizons, more queries, and higher computational cost. However, the query rate increases only moderately, indicating that the policy remains selective rather than asking for feedback at every opportunity. The results demonstrate that the proposed query policy maintains efficient and selective feedback acquisition even as task complexity and belief-space size increase.

C. User Study

We conducted a Wizard-of-Oz (WoZ) user study to evaluate how different query policies affect user performance, interaction time, workload, and situation awareness. While the system evaluation in Section V-B measures system performance itself, this study examines how query timing and query content shape the human side of collaboration. The central comparison is between action-level queries, which ask users to reason about what the robot should do next, and state-level queries, which ask users to verify task facts that can be judged from the interface.

1) Participants:

Twelve participants (7 female and 5 male) took part in the study. Participants ranged in age from 21 to 26 years ($M = 23.0$, $SD = 1.48$). Each participant received \$20 upon completion of the study.

2) Study Design and Conditions:

The study employed a within-subject design. Each participant completed ten trials in total, consisting of five waste sorting scenarios and five tomato harvesting scenarios. In each domain, participants experienced five interaction conditions: **All**, **No**, **Ours1**, **Ours2**, and **KnowNo**. **All** requested feedback whenever a query was available, representing continuous human supervision. **No** performed the task without requesting user feedback. **Ours1** used the proposed uncertainty-based state query policy with the same interaction logic as the system-level **Ours** policy. **Ours2** used the same state-level query interface and hypothesis-selection rule, with WoZ scenario scripts generated from a different transition-probability profile. This profile created a broader range of uncertain state-query situations while preserving the same interaction principle: users verify task-level state hypotheses used by the belief update. **KnowNo** served as an action-level query baseline, in which users were asked to select the robot's next action rather than verify a task-level state hypothesis. We implemented this baseline within the same WoZ scripts, scenes, and tablet interface used by the other conditions. The comparison therefore isolates query content: **KnowNo** asks users to choose among action-level alternatives, whereas **Ours1** and **Ours2** ask users to verify task-level state hypotheses.

Each domain consisted of five scenarios and five interaction conditions. The assignment between scenarios and conditions was randomized for each participant such that every participant experienced all five conditions exactly once per domain. This design prevented a particular condition from being consistently associated with a specific scenario. Detailed scenario descriptions are provided in Appendix D.

3) Materials and Procedure:

Participants interacted with the robot through a tablet-based interface. The interface displayed the current task state, visual sensing information, task-relevant objects, and robot-generated questions with selectable answer options. The waste sorting scenarios required participants to identify waste categories, including general waste, paper, cans, and plastic. The tomato harvesting scenarios required participants to identify tomato states and support harvesting decisions, such as whether a target tomato was present, ripe, unripe, or rotten.

Before the experiment, participants received an explanation of the study objectives, the two task domains, and the robot interaction process. They were then introduced to the tablet interface and completed practice interactions to become familiar with the available information and response procedure. During the main session, participants completed ten trials without assistance. If a participant selected an incorrect answer, the response was recorded as incorrect for evaluation purposes, but the scenario continued using the intended task state so that subsequent interactions were not affected by earlier mistakes. After each scenario, participants completed questionnaires regarding workload, fatigue, and situation awareness.

4) Measures and Analysis:

We analyzed the user study along four axes. First, task success measured whether participants provided all required responses correctly within a task. **No** was reported for questionnaire-based measures but omitted from task success because it contains no scorable user responses. Second, operation time measured the active user operation time after subtracting passive scenario video duration. **No** was also omitted from this analysis so that operation time reflects active interaction rather than the absence of interaction.

Third, subjective workload was measured using NASA Raw-TLX, and fatigue was measured using a self-reported fatigue item. Fourth, situation awareness was evaluated using SART and SAGAT-style questions. We further decomposed SAGAT into three levels: Lv1 measured perception of the current state, Lv2 measured comprehension of the task situation, and Lv3 measured projection of future task behavior.

For domain-level condition comparisons, we used Cochran's Q for binary task success and Friedman tests for continuous or ordinal measures. Post-hoc comparisons used exact McNemar tests for task success and exact Wilcoxon signed-rank tests for other measures, with Bonferroni correction. For SAGAT-level analysis, we compared Lv1, Lv2, and Lv3 within each domain-condition cell using Friedman tests and exact Wilcoxon signed-rank post-hoc tests with Bonferroni correction.

5) Results: Task Success and Operation Time:

Table V summarizes task success and operation time. In waste sorting, the condition effect on task success was significant, Cochran's $Q(3) = 27.69$, $p < .001$, Kendall's $W = 0.77$. **Ours1** achieved 91.7% success and **Ours2** achieved 100.0%, matching or exceeding **All** (91.7%). In contrast, **KnowNo** achieved only 8.3%. Post-hoc tests showed that **KnowNo** was significantly lower than **All** ($p = .012$), **Ours1** ($p = .012$), and **Ours2** ($p = .006$). In tomato harvesting, the condition effect was not significant, $Q(3) = 5.40$, $p = .143$, but **Ours1** and

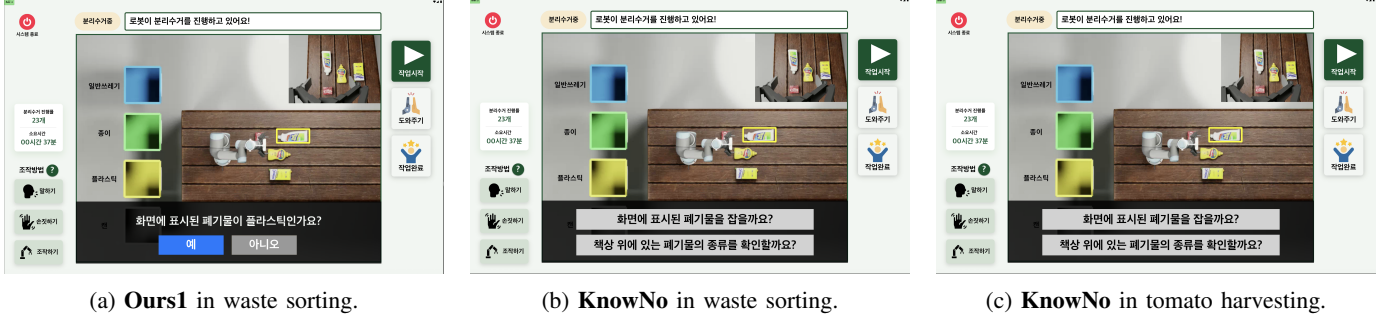


Fig. 8: Representative user-study interfaces. In **Ours1**, users answer an observable task-level state question, such as whether the highlighted waste object is plastic. In **KnowNo**, users may be asked to choose a robot action, such as whether to grasp the displayed object or inspect the object category. This comparison contrasts state-level verification with action-level decision making.

Ours2 both maintained 75.0% success, compared with 66.7% for **All**.

Operation time showed a similar pattern. In waste sorting, the condition effect was significant, $\chi^2(3) = 15.61$, $p = .002$, $W = 0.58$. **Ours1** (11.2 s) and **Ours2** (10.0 s) were much faster than **All** (30.0 s) and **KnowNo** (29.2 s). Post-hoc tests confirmed that both **Ours1** and **Ours2** were faster than **KnowNo** (both Bonferroni-adjusted $p = .023$). In tomato harvesting, the same direction was observed, with **Ours1** (21.4 s) and **Ours2** (17.6 s) faster than **All** (35.2 s), although the overall condition effect did not reach significance, $\chi^2(3) = 6.57$, $p = .085$.

These results show that query content shaped interaction efficiency. In the waste sorting **KnowNo** interface, users saw the object and bounding box, but were asked to decide whether the robot should grasp the displayed object or inspect the waste category. This action-level decision required users to infer why the planner needed another detection action. By contrast, **Ours1** asked users to judge the observable state itself, such as whether the highlighted object was plastic. Users could answer this task-level question directly from the current visual evidence, resulting in high success and short operation time. Tomato harvesting produced a complementary pattern: the **KnowNo** condition reached 100.0% task success because the tomato workflow made the next action more visually and temporally constrained after navigation, detection, picking, and scanning. Thus, action-level querying was easier to resolve in tomato harvesting than in waste sorting, where users had to decide whether hidden category uncertainty still justified another detect action.

6) Results: Workload and Situation Awareness:

Table VI summarizes workload and SART. NASA-RTLX showed significant condition effects in both waste sorting, $\chi^2(4) = 21.31$, $p < .001$, $W = 0.44$, and tomato harvesting, $\chi^2(4) = 16.31$, $p = .003$, $W = 0.34$. In waste sorting, **Ours2** produced lower workload ($M = 1.89$) than **KnowNo** ($M = 2.54$; Bonferroni-adjusted $p = .010$). In tomato harvesting, **Ours1** ($M = 1.89$) and **Ours2** ($M = 1.81$) both reduced workload relative to **All** ($M = 2.21$), with Bonferroni-adjusted $p = .020$ and $p = .039$, respectively. Fatigue did not show a significant condition effect, and SART did not

differ significantly across conditions in either domain: waste, $\chi^2(4) = 5.06$, $p = .281$; tomato, $\chi^2(4) = 5.24$, $p = .264$.

The workload and SART results show the main user-facing benefit of selective state queries: participants spent less active time and reported lower workload while retaining comparable perceived situation awareness. **Ours1** and **Ours2** therefore reduced interaction cost without drawing users away from the task state they needed to monitor.

7) Results: SAGAT Levels:

The SAGAT-level analysis provides a finer-grained view of how different query strategies shaped situation awareness. Table VII reports Lv1, Lv2, and Lv3 accuracy for all conditions shown in the SAGAT level figures.

In waste sorting, **All** produced high Lv1 accuracy ($M = 0.92$), while Lv2 and Lv3 were lower ($M = 0.38$ for both). This profile reflects strong awareness of visible information under continuous supervision. In contrast, **Ours1** maintained a more balanced profile across Lv1, Lv2, and Lv3 (0.79, 0.63, and 0.75), suggesting that state-level uncertainty queries supported not only perception but also task comprehension and projection.

Ours2 showed a more specialized profile. In waste sorting, it kept Lv1 high (0.88) and Lv3 relatively high (0.63), while Lv2 was low (0.08), indicating that broader query exposure emphasized visible state recognition and future task tracking more than intermediate causal explanations of why a specific query was issued. In tomato harvesting, Lv3 reached 0.96, the highest value among all tomato conditions. The difference between Lv1 and Lv3 in tomato **Ours2** was significant after Bonferroni correction ($p = .047$). Thus, the two proposed variants produced different forms of situation awareness: **Ours1** supported a more balanced awareness profile, whereas **Ours2** particularly strengthened projection of future task behavior.

8) User Study Discussion:

The user-study results highlight a central design point: effective selective querying depends on matching the query to information users can judge from the interface. **Ours1** and **Ours2** used state-level questions grounded in observable task facts, supporting high success with shorter operation time and lower workload. **KnowNo** asked action-level questions, which shifted the interaction toward planner-state inference. In waste sorting, this placed the key decision on detect-versus-grasp

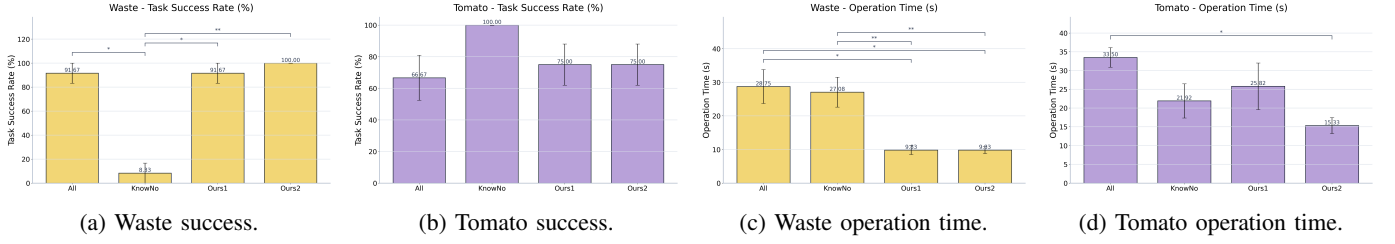


Fig. 9: Task success and operation time across query strategies. **No** is omitted from these two analyses so the panels focus on scorable user responses and active user operation. Error bars indicate standard error; brackets mark Bonferroni-corrected significant post-hoc comparisons.

TABLE V: Task success and operation time by domain and condition. Values are mean $\pm$ standard error. **No** is omitted because these measures focus on scorable user responses and active user operation.

Domain	Measure	All	Ours1	Ours2	KnowNo
Waste	Task success (%)	91.7 $\pm$ 8.3	91.7 $\pm$ 8.3	100.0 $\pm$ 0.0	8.3 $\pm$ 8.3
Waste	Operation time (s)	30.0 $\pm$ 6.7	11.2 $\pm$ 1.4	10.0 $\pm$ 1.3	29.2 $\pm$ 5.7
Tomato	Task success (%)	66.7 $\pm$ 14.2	75.0 $\pm$ 13.1	75.0 $\pm$ 13.1	100.0 $\pm$ 0.0
Tomato	Operation time (s)	35.2 $\pm$ 3.2	21.4 $\pm$ 3.5	17.6 $\pm$ 2.2	25.9 $\pm$ 5.4

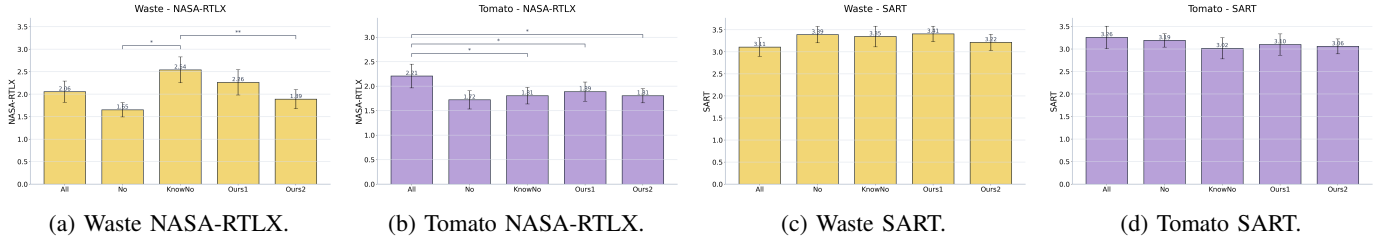


Fig. 10: Subjective workload and perceived situation awareness. NASA-RTLX is lower-is-better, whereas SART is higher-is-better. Error bars indicate standard error; brackets mark Bonferroni-corrected significant post-hoc comparisons.

TABLE VI: Subjective workload and SART by domain and condition. Values are mean $\pm$ standard error. Lower NASA-RTLX indicates lower workload; higher SART indicates higher perceived situation awareness.

Domain	Measure	All	No	Ours1	Ours2	KnowNo
Waste	NASA-RTLX	2.06 $\pm$ 0.24	1.65 $\pm$ 0.16	2.26 $\pm$ 0.28	1.89 $\pm$ 0.21	2.54 $\pm$ 0.29
Waste	SART	3.11 $\pm$ 0.21	3.39 $\pm$ 0.19	3.41 $\pm$ 0.18	3.22 $\pm$ 0.19	3.35 $\pm$ 0.23
Tomato	NASA-RTLX	2.21 $\pm$ 0.24	1.72 $\pm$ 0.19	1.89 $\pm$ 0.20	1.81 $\pm$ 0.15	1.81 $\pm$ 0.17
Tomato	SART	3.26 $\pm$ 0.25	3.19 $\pm$ 0.15	3.10 $\pm$ 0.24	3.06 $\pm$ 0.17	3.02 $\pm$ 0.23

selection when the visual scene looked actionable but the planner still needed category evidence.

The **KnowNo** cases also clarify the role of transparency. Visual highlights and scene views are useful when they align with the decision users are asked to make. In waste sorting, the **KnowNo** display showed the object and its bounding box, making the object salient without directly revealing the planner’s category uncertainty. State-level queries align the visual cue and requested judgment more directly: the interface highlights the object, and the question asks users to verify the task fact associated with that object.

The SAGAT results further show that query strategies shape the structure of situation awareness. **Ours1** produced relatively balanced Lv1–Lv3 performance, supporting both current-state understanding and future-action projection. **Ours2**, especially in tomato harvesting, produced high Lv3 accuracy, suggesting that its broader query distribution encouraged users to follow

the overall task flow. These patterns imply that situation awareness is not determined only by how much information is shown; it also depends on what kind of question the system asks.

Overall, effective selective querying is a design strategy for placing human attention at belief-relevant moments. A useful query is grounded in information that users can interpret from the interface while still being informative for the robot’s planner. When this alignment holds, human feedback improves task performance, reduces cognitive burden, and shapes situation awareness in a task-relevant way.

The present user study provides a controlled first evaluation with 12 participants in a narrow age range of 21–26 years. This setting allowed us to compare all conditions within participant and keep scenario timing consistent across query policies. A natural next step is to run the same query policies in fully autonomous long-horizon deployments with broader

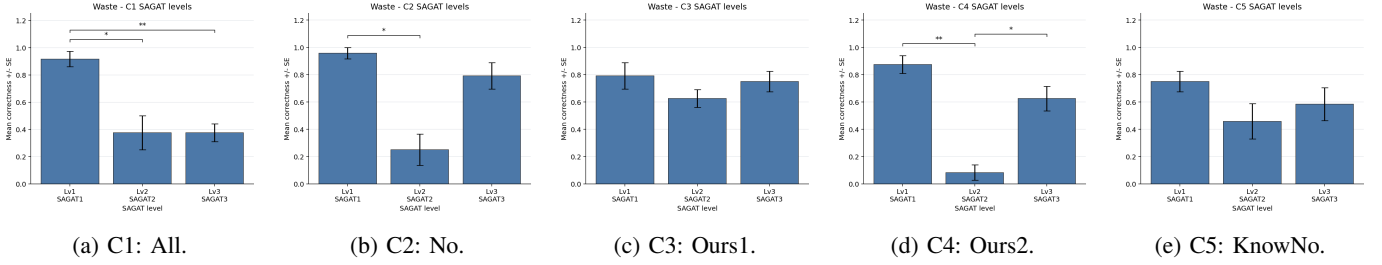


Fig. 11: SAGAT level profiles in waste sorting. Each panel compares Lv1, Lv2, and Lv3 within a condition. Error bars indicate standard error; brackets mark Bonferroni-corrected significant post-hoc comparisons.

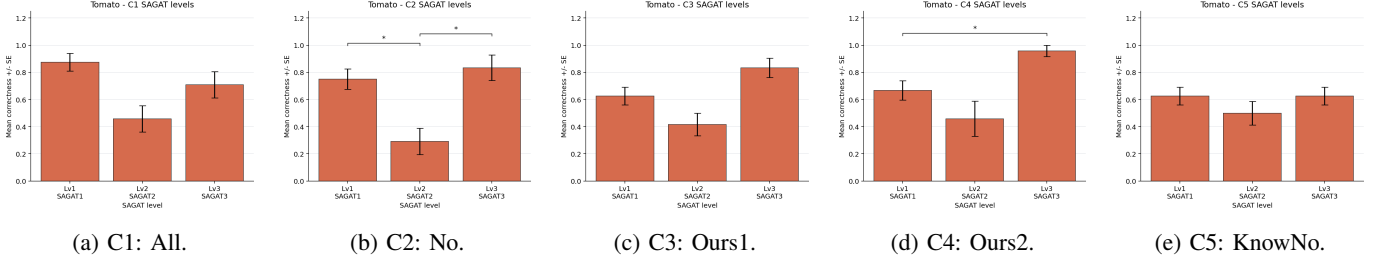


Fig. 12: SAGAT level profiles in tomato harvesting. Each panel compares Lv1, Lv2, and Lv3 within a condition. Error bars indicate standard error; brackets mark Bonferroni-corrected significant post-hoc comparisons.

TABLE VII: SAGAT level accuracy by domain and condition. Values are mean $\pm$ standard error. Lv1, Lv2, and Lv3 correspond to perception, comprehension, and projection, respectively.

Domain	Condition	Lv1	Lv2	Lv3
Waste	All	0.92 $\pm$ 0.06	0.38 $\pm$ 0.13	0.38 $\pm$ 0.07
Waste	No	0.96 $\pm$ 0.04	0.25 $\pm$ 0.12	0.79 $\pm$ 0.10
Waste	Ours1	0.79 $\pm$ 0.10	0.63 $\pm$ 0.07	0.75 $\pm$ 0.08
Waste	Ours2	0.88 $\pm$ 0.07	0.08 $\pm$ 0.06	0.63 $\pm$ 0.09
Waste	KnowNo	0.75 $\pm$ 0.08	0.46 $\pm$ 0.13	0.58 $\pm$ 0.12
Tomato	All	0.88 $\pm$ 0.07	0.46 $\pm$ 0.10	0.71 $\pm$ 0.10
Tomato	No	0.75 $\pm$ 0.08	0.29 $\pm$ 0.10	0.83 $\pm$ 0.09
Tomato	Ours1	0.63 $\pm$ 0.07	0.42 $\pm$ 0.08	0.83 $\pm$ 0.07
Tomato	Ours2	0.67 $\pm$ 0.07	0.46 $\pm$ 0.13	0.96 $\pm$ 0.04
Tomato	KnowNo	0.63 $\pm$ 0.07	0.50 $\pm$ 0.09	0.63 $\pm$ 0.07

participant groups.

VI. CONCLUSION AND FUTURE WORK

In this paper, we proposed a robotic decision-making framework that integrates symbolic knowledge and uncertainty, with the goal of reducing user workload. The proposed method maintains a belief over possible worlds within a POMDP-based planning framework and quantitatively models uncertainty through a probability distribution over these worlds. Based on this, the robot can assess its own uncertainty and selectively query the human only when necessary. This approach enables stable task execution without continuous supervision and minimizes unnecessary interactions, thereby reducing user intervention.

In particular, we leverage entropy-based confidence to determine when to issue queries and select the most informative hypothesis to construct questions, improving the efficiency of interaction. Experimental results show that the proposed method reduces the number of human interactions while

maintaining or improving task success rates, and effectively lowers user workload. User studies further support this trend, demonstrating that a system which actively manages uncertainty and requests assistance only when needed can operate effectively in real-world settings.

These results point to several extensions. First, richer feedback channels can build on the current binary hypothesis interface by allowing users to provide corrected symbolic facts or natural-language clarifications. Second, scalable frontier construction can extend the approach to larger domains through pruning, abstraction, or learned proposal distributions. Third, user-aware query selection can combine robot belief uncertainty with estimates of what users can judge from the interface. Finally, the framework can be extended to multi-robot settings, where multiple agents share uncertainty and coordinate when and whom to query. These extensions preserve the central idea of the framework: human feedback is most useful when it is requested at belief-relevant moments and expressed in a form that supports both planning and human

understanding.

ACKNOWLEDGEMENT

This research was supported in part by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT)(No. RS-2024-00461583) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00411007).

REFERENCES

- [1] K. N. Kaipa, A. S. Kankanhalli-Nagendra, N. B. Kumbala, S. Shriyam, S. S. Thevendria-Karthic, J. A. Marvel, and S. K. Gupta, "Addressing perception uncertainty induced failure modes in robotic bin-picking," *Robotics and Computer-Integrated Manufacturing*, vol. 42, pp. 17–38, 2016.
- [2] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.
- [3] H. Kurniawati and V. Yadav, "An online pomdp solver for uncertainty planning in dynamic environment," in *Robotics Research: The 16th International Symposium ISRR*. Springer, 2016, pp. 611–629.
- [4] A. D. Dragan, K. C. Lee, and S. S. Srinivasa, "Legibility and predictability of robot motion," in *International Conference on Human-Robot Interaction (HRI)*. IEEE, 2013, pp. 301–308.
- [5] M. A. Goodrich and A. C. Schultz, "Human-robot interaction: a survey," *Foundations and trends® in human-computer interaction*, vol. 1, no. 3, pp. 203–275, 2008.
- [6] J. Y. Chen and M. J. Barnes, "Human-agent teaming for multirobot control: A review of human factors issues," *IEEE Transactions on Human-Machine Systems*, vol. 44, no. 1, pp. 13–29, 2014.
- [7] D. Sadigh, A. D. Dragan, S. S. Sastry, and S. A. Seshia, "Active preference-based learning of reward functions," in *Proceedings of Robotics: Science and Systems (RSS)*, 2017. [Online]. Available: <https://escholarship.org/uc/item/88k894w7>
- [8] J. Leusmann, C. Wang, M. Gienger, A. Schmidt, and S. Mayer, "Understanding the uncertainty loop of human-robot interaction," *arXiv preprint arXiv:2303.07889*, 2023.
- [9] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, Z. Xu, D. Sadigh, A. Zeng, and A. Majumdar, "Robots that ask for help: Uncertainty alignment for large language model planners," in *Proceedings of the Conference on Robot Learning (CoRL)*, 2023.
- [10] K. Liang, Z. Zhang, and J. F. Fisac, "Introspective planning: Aligning robots' uncertainty with inherent task ambiguity," *Advances in Neural Information Processing Systems*, vol. 37, pp. 71 998–72 031, 2024.
- [11] J. F. Mullen and D. Manocha, "Lbap: Improved uncertainty alignment of llm planners using bayesian inference," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 18 716–18 723.
- [12] R. Davis, H. Shrobe, and P. Szolovits, "What is a knowledge representation?" *AI magazine*, vol. 14, no. 1, pp. 17–17, 1993.
- [13] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.
- [14] M. Tenorth and M. Beetz, "Knowledge processing for autonomous robot control," in *Proc. of AAAI Spring Symposium: Designing Intelligent Robots*, 2012.
- [15] Z. Liu, M. Jiang, and H. Lin, "A graph-based spatial temporal logic for knowledge representation and automated reasoning in cognitive robots," 2020.
- [16] L. Kunze, K. K. Doreswamy, and N. Hawes, "Using qualitative spatial relations for indirect object search," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 163–168.
- [17] S. Hasegawa, A. Taniguchi, Y. Hagiwara, L. El Hafi, and T. Taniguchi, "Inferring place-object relationships by integrating probabilistic logic and multimodal spatial concepts," in *Proc. of IEEE/SICE International Symposium on System Integration (SII)*, 2023, pp. 1–8.
- [18] T. Mota, M. Sridharan, and A. Leonardis, "Integrated commonsense reasoning and deep learning for transparent decision making in robotics," *SN Computer Science*, vol. 2, no. 4, p. 242, 2021.
- [19] S. Zhang, P. Khandelwal, and P. Stone, "iCORPP: Interleaved commonsense reasoning and probabilistic planning on robots," *Robotics and Autonomous Systems*, vol. 174, p. 104613, 2024.
- [20] A. Daruna, W. Liu, Z. Kira, and S. Chetnova, "RoboCSE: Robot common sense embedding," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2019, pp. 9777–9783.
- [21] A. Mitrevsk, P. G. Plöger, and G. Lakemeyer, "Ontology-assisted generalisation of robot action execution knowledge," in *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 6763–6770.
- [22] A. Olivares-Alarcos, D. Beßler, A. Khamis, P. Goncalves, M. K. Habib, J. Bermejo-Alonso, M. Barreto, M. Diab, J. Rosell, J. Quintas et al., "A review and comparison of ontology-based approaches to robot autonomy," *The Knowledge Engineering Review*, vol. 34, p. e29, 2019.
- [23] S. Manzoor, Y. G. Rocha, S.-H. Joo, S.-H. Bae, E.-J. Kim, K.-J. Joo, and T.-Y. Kuc, "Ontology-based knowledge representation in robotic systems: A survey oriented toward applications," *Applied Sciences*, vol. 11, no. 10, p. 4324, 2021.
- [24] C. Aeronautiques, A. Howe, C. Knoblock, I. D. McDermott, A. Ram, M. Veloso, D. Weld, D. W. Sri, A. Barrett, D. Christianson et al., "PDDL—the planning domain definition language," *Technical Report*, Tech. Rep., 1998.
- [25] M. Helmert, "Concise finite-domain representations for pddl planning tasks," *Artificial Intelligence*, vol. 173, no. 5-6, pp. 503–535, 2009.
- [26] J. Hoffmann, "Ff: The fast-forward planning system," *AI magazine*, vol. 22, no. 3, pp. 57–57, 2001.
- [27] M. Helmert, "The fast downward planning system," *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.
- [28] A. B. Corrêa and G. De Giacomo, "Lifted planning: Recent advances in planning using first-order representations," in *IJCAI*, 2024, pp. 8010–8019.
- [29] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman et al., "Do as i can, not as i say: Grounding language in robotic affordances," 2022.
- [30] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar et al., "Inner monologue: Embodied reasoning through planning with language models," 2022.
- [31] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, "ProgPrompt: Program generation for situated robot task planning using large language models," *Autonomous Robots*, vol. 47, no. 8, pp. 999–1012, 2023.
- [32] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, "Code as policies: Language model programs for embodied control," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 9493–9500.
- [33] Y. Ding, X. Zhang, C. Paxton, and S. Zhang, "Task and motion planning with large language models for object rearrangement," in *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023, pp. 2086–2092.
- [34] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa et al., "Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 5021–5028.
- [35] A. Leanza, A. Moroncelli, G. Vizzari, F. Braghin, L. Roveda, and B. Spahiu, "Conceptbot: Enhancing robot's autonomy through task decomposition with large language models and knowledge graph," *arXiv preprint arXiv:2509.00570*, 2025.
- [36] S. Singh, K. Swaminathan, N. Dash, R. Singh, S. Banerjee, M. Sridharan, and M. Krishna, "Adaptbot: Combining llm with knowledge graphs and human input for generic-to-specific task decomposition and knowledge refinement," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4345–4351.
- [37] J. Zhou, J. Miao, X. Wang, and J. Yu, "Enhancing llm planning for robotics manipulation through hierarchical procedural knowledge graphs," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [38] W. Xu, M. Wang, W. Zhou, and H. Li, "P-RAG: Progressive retrieval augmented generation for planning on embodied everyday task," in *Proc. of ACM International Conference on Multimedia (MM)*, 2024, pp. 6969–6978.
- [39] Z. Wang, S. X. Teo, J. J. Chew, and W. Shi, "InstructRAG: Leveraging retrieval-augmented generation on instruction graphs for llm-based task planning," in *Proc. of International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2025.
- [40] N. Kumar, W. Shen, F. Ramos, D. Fox, T. Lozano-Pérez, L. P. Kaelbling, and C. R. Garrett, "Open-world task and motion planning via vision-language model generated constraints," *IEEE Robotics and Automation Letters*, 2026.

- [41] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3d value maps for robotic manipulation with language models," in *Proc. of Conference on Robot Learning (CoRL)*, 2023, pp. 540–562.
- [42] Z. Dai, A. Asgharivaskasi, T. Duong, S. Lin, M.-E. Tzes, G. Pappas, and N. Atanasov, "Optimal scene graph planning with large language model guidance," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14 062–14 069.
- [43] Y. Liu, L. Palmieri, S. Koch, I. Georgievski, and M. Aiello, "DELTA: Decomposed efficient long-term robot task planning using large language models," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 10 995–11 001.
- [44] D. Paulius, A. Agostini, B. Quartey, and G. Konidaris, "Bootstrapping object-level planning with large language models," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 16 233–16 239.
- [45] B. Quartey, E. Rosen, S. Tellex, and G. Konidaris, "Verifiably following complex robot instructions with foundation models," in *Proc. of IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 1–8.
- [46] A. Somani, N. Ye, D. Hsu, and W. S. Lee, "Despot: Online pomdp planning with regularization," *Advances in neural information processing systems*, vol. 26, 2013.
- [47] H. Bai, S. Cai, N. Ye, D. Hsu, and W. S. Lee, "Intention-aware online pomdp planning for autonomous driving in a crowd," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2015, pp. 454–460.
- [48] M. Cramer, K. Kellens, and E. Demeester, "Probabilistic decision model for adaptive task planning in human-robot collaborative assembly based on designer and operator intents," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7325–7332, 2021.
- [49] R. Z. B. Bravo, A. Leiras, and F. L. Cyrino Oliveira, "The use of UAVs in humanitarian relief: An application of pomdp-based methodology for finding victims," *Production and Operations Management*, vol. 28, no. 2, pp. 421–440, 2019.
- [50] M. Lauri and R. Ritala, "Planning for robotic exploration based on forward simulation," *Robotics and Autonomous Systems*, vol. 83, pp. 15–31, 2016.
- [51] S.-K. Kim, O. Salzman, G. Agarwal et al., "PLGRIM: Hierarchical value learning for large-scale exploration in unknown environments," in *Proceedings of the 31st International Conference on Automated Planning and Scheduling (ICAPS)*, vol. 31, 2021, pp. 652–662.
- [52] F. Vanegas, D. Campbell, M. Eich, and F. Gonzalez, "UAV based target finding and tracking in GPS-denied and cluttered environments," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 2307–2313.
- [53] Y. Xiao, S. Katt, A. ten Pas, S. Chen, and C. Amato, "Online planning for target object search in clutter under partial observability," in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8241–8247.
- [54] J. Pajarinen, J. Lundell, and V. Kyrki, "POMDP planning under object composition uncertainty: Application to robotic manipulation," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 41–56, 2023.
- [55] M. Li, Z. Zhang, and B. Song, "Uncertainty-aware neural-symbolic task planning with balanced exploration and execution for domestic service robot," *Expert Systems with Applications*, p. 132962, 2026.
- [56] S. Sanner and K. Kersting, "Symbolic dynamic programming for first-order pomdps," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 24, no. 1, 2010, pp. 1140–1146.
- [57] S. Zhang and P. Stone, "Corpp: Commonsense reasoning and probabilistic planning, as applied to dialog with a mobile robot," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 29, no. 1, 2015.
- [58] S. A. Serrano, E. Santiago, J. Martinez-Carranza, E. F. Morales, and L. E. Sucar, "Knowledge-based hierarchical pomdps for task planning," *Journal of Intelligent & Robotic Systems*, vol. 101, no. 4, p. 82, 2021.
- [59] W. Tang, X. He, Y. Huang, Y. Xiao, C. Lu, and P. Cai, "Tru-pomdp: Task planning under uncertainty via tree of hypotheses and open-ended pomdps," *arXiv preprint arXiv:2506.02860*, 2025.
- [60] P. Scharre, "Introduction to autonomy in weapon systems," 2015.
- [61] S. Nahavandi, "Trusted autonomy between humans and robots: Toward human-on-the-loop in robotics and autonomous systems," *IEEE Systems, Man, and Cybernetics Magazine*, vol. 3, no. 1, pp. 10–17, 2017.
- [62] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, "Hg-dagger: Interactive imitation learning with human experts," in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 8077–8083.
- [63] A. Mandelkar, D. Xu, R. Martín-Martín, Y. Zhu, L. Fei-Fei, and S. Savarese, "Human-in-the-loop imitation learning using remote teleoperation," *arXiv preprint arXiv:2012.06733*, 2020.
- [64] R. Brachman and H. Levesque, *Knowledge representation and reasoning*. Elsevier, 2004.
- [65] S. Russell, P. Norvig, and A. Intelligence, "A modern approach," *Artificial Intelligence*. Prentice-Hall, Englewood Cliffs, vol. 25, no. 27, pp. 79–80, 1995.
- [66] J. Johnson, R. Krishna, M. Stark, L.-J. Li, D. Shamma, M. Bernstein, and L. Fei-Fei, "Image retrieval using scene graphs," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 3668–3678.
- [67] I. Armeni, Z.-Y. He, J. Gwak, A. R. Zamir, M. Fischer, J. Malik, and S. Savarese, "3d scene graph: A structure for unified semantics, 3d space, and camera," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 5664–5673.
- [68] F. Yang, D. Lyu, B. Liu, and S. Gustafson, "PEORL: Integrating symbolic planning and hierarchical reinforcement learning for robust decision-making," 2018.
- [69] A. Bouguerra, L. Karlsson, and A. Saffiotti, "Monitoring the execution of robot plans using semantic knowledge," *Robotics and Autonomous Systems*, vol. 56, no. 11, pp. 942–954, 2008.
- [70] J. Pineau, G. Gordon, S. Thrun et al., "Point-based value iteration: An anytime algorithm for pomdps," in *Ijcai*, vol. 3, 2003, pp. 1025–1032.
- [71] S. Ross, J. Pineau, S. Paquet, and B. Chaib-Draa, "Online planning algorithms for pomdps," *Journal of Artificial Intelligence Research*, vol. 32, pp. 663–704, 2008.
- [72] D. Silver and J. Veness, "Monte-carlo planning in large pomdps," *Advances in neural information processing systems*, vol. 23, 2010.
- [73] P. Doshi and P. J. Gmytrasiewicz, "A particle filtering based approach to approximating interactive pomdps," in *AAAI*, 2005, pp. 969–974.
- [74] M. H. Lim, T. J. Becker, M. J. Kochenderfer, C. J. Tomlin, and Z. N. Sunberg, "Optimality guarantees for particle belief approximation of pomdps," *Journal of Artificial Intelligence Research*, vol. 77, pp. 1591–1636, 2023.
- [75] Z. N. Sunberg and M. J. Kochenderfer, "Online algorithms for POMDPs with continuous state, action, and observation spaces," in *Proceedings of the 28th International Conference on Automated Planning and Scheduling (ICAPS)*, 2018, pp. 259–263.
- [76] S. Brechtel, T. Gindele, and R. Dillmann, "Probabilistic decision-making under uncertainty for autonomous driving using continuous POMDPs," in *Proceedings of the IEEE International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2014, pp. 392–399.
- [77] H. Bai, D. Hsu, M. J. Kochenderfer, and W. S. Lee, "Unmanned aircraft collision avoidance using continuous-state POMDPs," in *Proceedings of Robotics: Science and Systems (RSS)*, 2011, pp. 1–8.
- [78] J. Pajarinen and V. Kyrki, "Robotic manipulation of multiple objects as a POMDP," *Artificial Intelligence*, vol. 247, pp. 213–228, 2017.
- [79] E. A. Hansen and Z. Feng, "Dynamic programming for pomdps using a factored state representation," in *AIPS*, 2000, pp. 130–139.
- [80] Z. Feng and E. Hansen, "Approximate planning for factored pomdps," in *Proceedings of the 6th European Conference on Planning*, 2001.
- [81] T. Veiga, M. Spaan, and P. Lima, "Point-based pomdp solving with factored value function approximation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, no. 1, 2014.
- [82] S. R. Eddy, "Hidden markov models," *Current opinion in structural biology*, vol. 6, no. 3, pp. 361–365, 1996.
- [83] K. Murphy and Y. Weiss, "The factored frontier algorithm for approximate inference in dbns," *arXiv preprint arXiv:1301.2296*, 2013.
- [84] M. Ghallab, D. Nau, and P. Traverso, *Automated planning and acting*. Cambridge University Press, 2016.
- [85] V. Lifschitz, *Answer set programming*. Springer Cham, 2019, vol. 3.
- [86] M. Richardson and P. Domingos, "Markov logic networks," *Machine learning*, vol. 62, no. 1, pp. 107–136, 2006.
- [87] M. Qu and J. Tang, "Probabilistic logic neural networks for reasoning," *Advances in neural information processing systems*, vol. 32, 2019.
- [88] H. Kurniawati, D. Hsu, W. S. Lee et al., "Sarsop: Efficient point-based pomdp planning by approximating optimally reachable belief spaces," in *Robotics: Science and systems*, vol. 2008. Zurich, Switzerland, 2008.
- [89] M. R. Endsley, "Automation and situation awareness," in *Automation and human performance*. CRC Press, 2018, pp. 163–181.
- [90] D. Davies and R. Parasuraman, *The psychology of violence*. Academic Press, New York, 1982, vol. 1.

- [91] D. R. Davies, G. Matthews, R. B. Stammers, and S. J. Westerman, *Human performance: Cognition, stress and individual differences*. Psychology Press, 2013.
- [92] A. Mack, “Inattention blindness: Looking without seeing,” *Current directions in psychological science*, vol. 12, no. 5, pp. 180–184, 2003.
- [93] C. Y. Wong and G. Seet, “Workload, awareness and automation in multiple-robot supervision,” *International Journal of Advanced Robotic Systems*, vol. 14, no. 3, p. 1729881417710463, 2017.
- [94] S. M. Galster, B. A. Knott, and R. D. Brown, “Managing multiple uavs: Are we asking the right questions?” in *Proceedings of the human factors and ergonomics society annual meeting*, vol. 50, no. 3. SAGE Publications Sage CA: Los Angeles, CA, 2006, pp. 545–549.
- [95] J. Baraglia, M. Cakmak, Y. Nagai, R. P. Rao, and M. Asada, “Efficient human-robot collaboration: when should a robot take initiative?” *The International Journal of Robotics Research*, vol. 36, no. 5-7, pp. 563–579, 2017.
- [96] A. Murata, “Ergonomics and cognitive engineering for robot-human cooperation,” in *Proceedings 9th IEEE International Workshop on Robot and Human Interactive Communication. IEEE RO-MAN 2000*. IEEE, 2000, pp. 206–211.
- [97] G. Borragán, H. Slama, M. Bartolomei, and P. Peigneux, “Cognitive fatigue: A time-based resource-sharing account,” *Cortex*, vol. 89, pp. 71–84, 2017.
- [98] A. Rajavenkatanarayanan, V. Kanak, K. Tsiakas, J. Brady, D. Calderon, G. Wylie, and F. Makedon, “Towards a robot-based multimodal framework to assess the impact of fatigue on user behavior and performance: A pilot study,” in *Proceedings of the 12th ACM International Conference on Pervasive Technologies Related to Assistive Environments*, 2019, pp. 493–498.
- [99] S. Chitta, I. Sucan, and S. Cousins, “Moveit![ros topics],” *IEEE robotics & automation magazine*, vol. 19, no. 1, pp. 18–19, 2012.
- [100] Y. Tian, Q. Ye, and D. Doermann, “Yolov12: Attention-centric real-time object detectors,” *arXiv preprint arXiv:2502.12524*, 2025.
- [101] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat et al., “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.

Algorithm 2: POMCP Search with Applicable Actions

```

1 Input: belief  $b$ 
2 Output: selected action  $a \in \mathcal{A}$ 
3 procedure SEARCH( $b$ )
4    $h \leftarrow$  root history
5    $s^K \leftarrow \text{knowledge}(b)$ 
6   repeat
7     if  $B(h) \neq \emptyset$  then
8       | sample  $s \sim B(h)$ 
9     else
10      |  $s \leftarrow s^K$ 
11     SIMULATE( $s, h, 0$ )
12   until TIMEOUT;
13    $a \leftarrow \text{SEARCH-BEST}(h, s^K, \text{false})$ 
14   return  $a$ 

```

APPENDIX CONTENTS

This appendix provides supplementary materials for the planning algorithms, experimental settings, additional results, and user-study materials used in the main text.

- 1) Appendix A. Planning algorithms
- 2) Appendix B. Detailed experimental settings
- 3) Appendix C. Additional experimental results
- 4) Appendix D. User-study scenarios, condition assignment, questionnaires, and answer summaries

APPENDIX A PLANNING ALGORITHMS

This appendix provides the detailed pseudocode for the POMCP-based search routines described in Sec. IV-B2.

APPENDIX B DETAILED EXPERIMENTAL SETTINGS

This appendix summarizes implementation details that support the experimental setup in Sec. V-A. Both domains use five scenario files, `scene_01.yaml`–`scene_05.yaml`. For each threshold or policy condition, we evaluate all five scenarios with 200 randomized episodes per scenario. The maximum episode length is 50 steps, and all POMCP searches use 100 simulations per planning step, maximum simulation depth 20, discount factor $\gamma = 0.95$, and UCB exploration constant $c = 1.0$.

The waste-sorting detect category transition probability of 0.50 defines the controlled category-ambiguity profile used in the experiments. It creates competing category hypotheses after detection, allowing the planner and query policy to be evaluated in a setting where category uncertainty matters for bin placement. The observation model is specified separately from this transition uncertainty; for example, the waste detect category observation probability is 0.80 and is used when computing the likelihood term $O(o \mid s_k^*)$ during belief update.

Algorithm 3: POMCP Simulation with Applicable Actions

```

1 Input: state  $s$ , history  $h$ , depth  $depth$ 
2 Output: simulated return  $R$ 
3 procedure SIMULATE ( $s, h, depth$ )
4   if  $\gamma^{depth} < \epsilon$  then
5     return 0
6   if there is no  $a[s] \in \mathcal{A}$  then
7      $N(h) \leftarrow N(h) + 1$ 
8     return 0
9   if  $h \notin T$  then
10    foreach  $a[s] \in \mathcal{A}$  do
11       $T, N(ha), V(ha) \leftarrow T \cup \{ha\}, 0, 0$ 
12       $N(h) \leftarrow N(h) + 1$ 
13      return ROLLOUT ( $s, depth$ )
14    $a \leftarrow \text{SEARCH-BEST}(h, s, \text{true})$ 
15   sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
16    $R \leftarrow r + \gamma \cdot \text{SIMULATE}(s', ha, depth + 1)$ 
17    $B(h) \leftarrow B(h) \cup \{s\}$ 
18    $N(h) \leftarrow N(h) + 1$ 
19    $N(ha) \leftarrow N(ha) + 1$ 
20    $V(ha) \leftarrow V(ha) + \frac{R - V(ha)}{N(ha)}$ 
21   return  $R$ 

```

Algorithm 4: Action Selection and Rollout

```

1 Input: history  $h$ , state  $s$ 
2 procedure SEARCH-BEST ( $h, s, use\_ucb$ )
3    $\mathcal{C} \leftarrow \{a \mid a[s] \in \mathcal{A}, ha \in T\}$ 
4   if  $\mathcal{C} = \emptyset$  then
5     return  $\emptyset$ 
6   if  $use\_ucb$  then
7     return  $\arg \max_{a \in \mathcal{C}} V(ha) + c \cdot \sqrt{\frac{\log N(h)}{1 + N(ha)}}$ 
8   else
9     return  $\arg \max_{a \in \mathcal{C}} V(ha)$ 
10 procedure ROLLOUT ( $s, depth$ )
11   if  $(\gamma^{depth} < \epsilon \vee \gamma = 0) \wedge depth > 0$  then
12     return 0
13   if there is no  $a[s] \in \mathcal{A}$  then
14     return 0
15    $a \sim \text{Uniform}(\{a \mid a[s] \in \mathcal{A}\})$ 
16   sample  $(s', o, r)$  from simulator  $\mathcal{G}(s, a)$ 
17   return  $r + \gamma \cdot \text{ROLLOUT}(s', depth + 1)$ 

```

For each episode, the runner generates and records a pseudorandom seed. The seed is passed to the execution script and controls Python `random`, NumPy sampling, and the random-query policy when applicable. This allows individual raw episodes to be traced back to the scenario, threshold or policy condition, and sampled execution trajectory.

TABLE VIII: Planner and evaluation parameters used in the system experiments.

Parameter	Value
POMCP simulations per planning step	100
Maximum simulation depth	20
Discount factor γ	0.95
UCB exploration constant c	1.0
Maximum episode steps	50
Scenarios per domain	5
Episodes per scenario-condition	200
Episodes per domain-condition	1000

TABLE IX: Domain model probabilities used in the reported experiments.

Domain	Model component	Probability
Waste sorting	Detect observed transition	0.90
Waste sorting	Detect category transition	0.50
Waste sorting	Pick transition	0.90
Waste sorting	Place transition	0.90
Waste sorting	Detect observed observation	0.90
Waste sorting	Detect category observation	0.80
Waste sorting	Pick/place observation	0.95
Tomato harvesting	Navigate transition	0.90
Tomato harvesting	Detect transition	0.95
Tomato harvesting	Pick transition	0.95
Tomato harvesting	Scan transition	0.90
Tomato harvesting	Place/discard transition	0.99
Tomato harvesting	Detect observed observation	0.85
Tomato harvesting	Detect classification observation	0.95
Tomato harvesting	Scan observation	0.85

APPENDIX C ADDITIONAL EXPERIMENTAL RESULTS

The main text reports aggregate results for the threshold sweep and policy comparison. This appendix summarizes the design logic behind these aggregate values.

Threshold sweep.: The threshold τ controls when belief confidence is considered sufficient for autonomous execution. Low thresholds allow the robot to commit to a state estimate with little feedback, which reduces query frequency and favors autonomy. Higher thresholds request more feedback and concentrate the belief before downstream actions are selected. This produces the saturation pattern in Table II: both domains improve sharply around $\tau = 0.7$, while larger values mainly increase query count after the task-relevant hypotheses have already been resolved.

Policy comparison.: The policy comparison in Table III uses $\tau = 0.8$ for **Ours**. **All** represents an upper-interaction baseline that asks whenever a query is available. **No** represents fully autonomous execution without corrective feedback. **Random** uses the same query interface but ignores belief confidence when deciding whether to ask. Thus, the comparison separates three effects: availability of human information, timing of feedback, and information gain of the selected hypothesis. The strong performance of **Ours** relative to **Random**, especially in tomato harvesting, shows the value of aligning feedback with the current belief ambiguity.

Domain difference.: Waste sorting contains short pick-and-place sequences and requires correct category assignment before placement. Tomato harvesting contains longer action chains in which ambiguous tomato state estimates affect later scan, place, and discard decisions. Consequently, resolving belief errors at the right moment has greater leverage in tomato harvesting, while waste sorting gives shorter opportunities for uncertainty to propagate.

APPENDIX D USER STUDY SCENARIOS AND INTERFACES

This appendix summarizes the Wizard-of-Oz user-study materials referenced in Sec. V-C. Each participant completed five waste sorting scenarios and five tomato harvesting scenarios. The mapping between scenarios and interaction conditions was randomized for each participant so that every participant experienced each condition exactly once per domain.

TABLE X: User-study scenario structure. Scenario identifiers were used for counterbalancing and do not imply a fixed condition assignment.

Domain	Scenario IDs	Primary variation
Waste sorting	W1–W5	Waste category, object order, and detection ambiguity
Tomato harvesting	T1–T5	Tomato state, stem location, and scan/harvest ambiguity

Waste sorting scenarios.: The waste sorting trials showed a tabletop scene with four waste objects and category-dependent target bins. Participants answered questions about visually highlighted objects or selected actions depending on the assigned condition. In state-level query conditions, the interface asked participants to verify observable task facts such as whether the highlighted object belonged to a specific waste category. In the **KnowNo** condition, participants selected the robot’s next action, which could require deciding whether the robot should inspect an object again or proceed to grasp and place it.

Tomato harvesting scenarios.: The tomato harvesting trials showed tomatoes attached to two stems and required participants to reason about whether tomatoes should be harvested, ignored, scanned, placed, or discarded. State-level query conditions asked participants to verify facts such as object existence or tomato condition. In **KnowNo**, participants selected action-level alternatives in the harvesting sequence. These action choices were generally more constrained than in waste sorting because the tomato workflow follows a clearer navigation–detection–pick–scan–place/discard order.

Questionnaire timing.: After each scenario, participants completed NASA Raw-TLX, fatigue, SART, and SAGAT-style items. SAGAT questions were grouped into three levels: Lv1 for perception of the current state, Lv2 for comprehension of the task situation, and Lv3 for projection of future robot behavior. The analyses in Sec. V-C6 and Sec. V-C7 use these post-scenario responses.

A. Condition Codes and Participant-Level Assignment

Table XI defines the condition codes used in the user-study logs. Table XII reports the participant-level trial assignment.

Each token has the form D_SxCy , where D is the domain (W : waste sorting, T : tomato harvesting), Sx is the scenario identifier, and Cy is the condition code.

TABLE XI: Condition-code definitions used in the user-study logs.

Code	Condition	Interaction policy
C1	All	Query whenever feedback is available
C2	No	No user feedback request
C3	Ours1	State-level uncertainty query policy
C4	Ours2	State-level query exposure condition with varied transition
C5	KnowNo	Action-level query baseline

B. Questionnaires

The questionnaires were administered in Korean. The following English translation lists the questionnaire content used in the user study.

SAGAT: tomato harvesting.: Q1. How many tomatoes are currently displayed on the screen? Options: 0, 1, 2, I do not know. Q2. What was the state of the tomato most recently processed by the robot? Options: ripe, unripe, damaged/rotten, I do not know. Q3. What is the key information that the user should judge in the current situation? Options: whether the target tomato is harvestable, whether the robot should move to the next location, whether there is information that the robot judged incorrectly, I do not know. Q4. Why is user confirmation needed in the current situation? Options: because the tomato ripeness judgment is uncertain, because the tomato damage judgment is uncertain, because a harvest target may have been missed, I do not know. Q5. If the user takes no action, what is the robot most likely to do? Options: remain waiting, continue the task according to the current judgment, terminate the task, I do not know. Q6. If the robot selected the wrong tomato and the user does not correct it, what is the most likely outcome? Options: the wrong tomato may be harvested, the robot will automatically select the correct tomato again, the task will terminate immediately, I do not know.

SAGAT: waste sorting.: Q1. How many objects are currently on the table in the screen? Options: 0, 1, 2 or more, I do not know. Q2. What was the material/category of the waste object most recently processed by the robot? Options: plastic, can, general waste, paper. Q3. What is the key information that the user should judge in the current situation? Options: whether the next object exists, whether the target object can be grasped from the current position, whether the target object was moved to the correct recycling bin, I do not know. Q4. Why was user confirmation needed in the previous situation? Options: because the target object’s waste category judgment is uncertain, because the target object is outside the robot’s reachable range, because the target object is contaminated and its recyclability should be judged, I do not know. Q5. If the user takes no action, what is the robot most likely to do? Options: remain waiting, continue the waste-sorting task according to the current judgment, terminate the task, I do not know. Q6. If the robot sorted an object incorrectly and the user does not correct the task, what is the most likely outcome?

TABLE XII: Participant-level condition assignment and trial order.

Participant	Tomato harvesting trials	Waste sorting trials
p001	T_S1C1, T_S4C4, T_S4C5, T_S2C3, T_S3C2	W_S4C5, W_S1C2, W_S5C1, W_S2C3, W_S3C4
p002	T_S4C5, T_S2C3, T_S1C1, T_S3C2, T_S5C4	W_S3C2, W_S4C3, W_S1C5, W_S2C4, W_S5C1
p003	T_S4C3, T_S2C1, T_S1C2, T_S5C4, T_S3C5	W_S4C4, W_S3C5, W_S1C1, W_S5C3, W_S2C2
p004	T_S5C1, T_S2C2, T_S1C3, T_S3C5, T_S4C4	W_S3C2, W_S4C1, W_S2C4, W_S5C3, W_S1C5
p005	T_S2C2, T_S5C4, T_S4C1, T_S3C3, T_S1C5	W_S1C2, W_S4C5, W_S2C4, W_S3C1, W_S5C3
p006	T_S5C2, T_S2C4, T_S3C5, T_S1C1, T_S4C3	W_S2C2, W_S3C3, W_S5C5, W_S1C4, W_S4C1
p007	T_S3C1, T_S5C4, T_S2C5, T_S1C2, T_S4C3	W_S4C5, W_S1C3, W_S2C4, W_S3C1, W_S5C2
p008	T_S1C5, T_S5C2, T_S4C1, T_S2C3, T_S3C4	W_S3C1, W_S1C4, W_S4C3, W_S2C5, W_S5C2
p009	T_S4C1, T_S5C3, T_S2C5, T_S1C4, T_S3C2	W_S3C4, W_S5C2, W_S1C1, W_S4C5, W_S2C3
p010	T_S2C1, T_S1C5, T_S5C3, T_S4C4, T_S3C2	W_S4C4, W_S2C2, W_S5C5, W_S1C1, W_S4C3
p011	T_S2C1, T_S1C5, T_S5C2, T_S4C4, T_S3C3	W_S2C2, W_S3C3, W_S1C1, W_S2C4, W_S4C5
p012	T_S4C2, T_S3C3, T_S2C5, T_S5C4, T_S1C1	W_S3C3, W_S5C5, W_S4C4, W_S1C1, W_S2C2

Options: the target object may be placed in the wrong bin, the robot will automatically recognize the error and sort it again, the robot will immediately terminate all waste sorting, I do not know.

NASA Raw-TLX.: N1. How mentally demanding was this task? N2. How physically demanding was this task? N3. How hurried or rushed did the task pace feel? N4. How successful did you feel in performing the requested task? N5. How hard did you have to work to achieve task success? N6. How insecure, discouraged, irritated, stressed, or annoyed did you feel during the task?

Cognitive fatigue.: F. What was your level of fatigue?

SART.: S1. How unstable or changeable was the situation? S2. How complicated was the situation? S3. How many variables were changing within the situation? S4. How alert were you in the current situation? S5. How focused were you on the current situation? S6. How divided was your attention in the current situation? S7. How much spare cognitive capacity did you have in the current situation? S8. How much information did you obtain about the current situation? S9. How good was the information you obtained about the current situation? S10. How familiar were you with the current situation?

C. Scenario-Level Answer Summary

Table XIII and Table XIV summarize the scenario-condition scoring records used for task-response correctness. The columns report the number of scored trials, the mean number of correct responses over the mean number of scorable responses, and the corresponding mean accuracy.

TABLE XIII: Tomato harvesting scenario-condition answer summary.

Scenario	Condition	<i>n</i>	Correct/Total	Accuracy
S1	C1: All	4	13.0/13.8	0.95
S1	C3: Ours1	1	8.0/8.0	1.00
S1	C4: Ours2	1	7.0/7.0	1.00
S1	C5: KnowNo	4	4.0/4.0	1.00
S2	C1: All	3	13.7/14.0	0.98
S2	C3: Ours1	3	6.3/7.0	0.90
S2	C4: Ours2	1	7.0/7.0	1.00
S2	C5: KnowNo	3	4.0/4.0	1.00
S3	C1: All	1	14.0/14.0	1.00
S3	C3: Ours1	3	6.7/7.0	0.95
S3	C4: Ours2	1	6.0/6.0	1.00
S3	C5: KnowNo	3	3.0/3.0	1.00
S4	C1: All	3	13.3/13.7	0.98
S4	C3: Ours1	3	7.3/8.0	0.92
S4	C4: Ours2	4	6.8/7.0	0.96
S4	C5: KnowNo	1	4.0/4.0	1.00
S5	C1: All	1	11.0/13.0	0.85
S5	C3: Ours1	2	7.0/7.0	1.00
S5	C4: Ours2	5	5.6/6.0	0.93
S5	C5: KnowNo	1	2.0/3.0	0.67

TABLE XIV: Waste sorting scenario-condition answer summary.

Scenario	Condition	<i>n</i>	Correct/Total	Accuracy
S1	C1: All	5	11.8/12.0	0.98
S1	C3: Ours1	1	5.0/5.0	1.00
S1	C4: Ours2	2	6.0/6.0	1.00
S1	C5: KnowNo	2	2.0/3.0	0.67
S2	C3: Ours1	2	5.0/5.0	1.00
S2	C4: Ours2	5	5.0/5.0	1.00
S2	C5: KnowNo	1	2.0/3.0	0.67
S3	C1: All	3	12.3/12.3	1.00
S3	C3: Ours1	3	5.0/5.0	1.00
S3	C4: Ours2	2	5.0/5.0	1.00
S3	C5: KnowNo	1	3.0/3.0	1.00
S4	C1: All	2	12.0/12.0	1.00
S4	C3: Ours1	3	4.3/5.0	0.87
S4	C4: Ours2	3	4.0/4.0	1.00
S4	C5: KnowNo	5	2.4/4.0	0.60
S5	C1: All	2	11.5/12.0	0.96
S5	C3: Ours1	3	5.0/5.0	1.00
S5	C5: KnowNo	3	3.0/4.0	0.75